

NATIONAL EMPLOYMENT VERIFICATION SYSTEM (NEVS)

A Blockchain-Based Trust Framework for Secure and Verifiable Digital Hiring

BY

Dhiraj Padhiyar 22C21043

Priyansh Dabhi 22C21016

Narendra Samanta 22C21093

Guided By

Prof. Madonna Lamin



A REPORT SUBMITTED TO

ITM SLS Baroda University

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

Computer Science & Engineering (with specialization in Information Technology)

School of Computer Science Engineering & Technology

April 2026

NATIONAL EMPLOYMENT VERIFICATION SYSTEM (NEVS)

A Blockchain-Based Trust Framework for Secure and Verifiable Digital Hiring

BY

Dhiraj Padhiyar 22C21043

Priyansh Dabhi 22C21016

Narendra Samanta 22C21093

Guided By

Prof. Madonna Lamin



A REPORT SUBMITTED TO

ITM SLS Baroda University

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

Computer Science & Engineering (with specialization in Information Technology)

School of Computer Science Engineering & Technology

April 2026

DECLARATION OF ORIGINALITY

The undersigned hereby declare that the report entitled "NATIONAL EMPLOYMENT VERIFICATION SYSTEM (NEVS) — A Blockchain-Based Trust Framework for Secure and Verifiable Digital Hiring" is the result of their own investigations and efforts. The report has not been accepted for any degree and is not being submitted concurrently in candidature for any degree or award.

Signature	:	1. _____ 2. _____ 3. _____
Name	:	1. <u>Narendra Samanta</u> 2. <u>Priyansh Dabhi</u> 3. <u>Dhiraj Padhiyar</u>
Date	:	_____

Date: _____

Name of Internal Guide: Prof. Madonna Lamin

Signature of Internal Guide: _____

Date: _____

Place: Vadodara, Gujarat, India

ACKNOWLEDGEMENTS

The authors would like to express their sincere gratitude and appreciation to their project supervisor, Prof. Madonna Lamin, whose guidance, encouragement, and technical expertise were instrumental throughout every phase of this project. Her insights into blockchain architecture and distributed systems substantially shaped the direction of the National Employment Verification System (NEVS).

The authors extend their heartfelt thanks to the Head of Department and the faculty of the School of Computer Science Engineering and Technology, ITM SLS Baroda University, for providing an environment conducive to research and innovation. The academic infrastructure, laboratory facilities, and mentorship provided by the institution were invaluable.

Gratitude is also extended to the Department Review Committee (DRC) for their constructive feedback during the continuous internal assessments, which helped refine the system architecture and implementation approach significantly.

Finally, the authors acknowledge the unwavering support of their families and friends, whose encouragement and patience throughout the duration of this project were deeply appreciated.

ABSTRACT

The National Employment Verification System (NEVS) is a decentralised, blockchain-based platform engineered to address critical challenges in modern digital hiring ecosystems, including fraudulent employment claims and unauthorised data exploitation. Leveraging Hyperledger Fabric, the system establishes a multi-organisation trust framework where employment records are validated through mutual consent and cryptographic endorsement.

The architecture integrates advanced enterprise features such as Chaincode-as-a-Service (CCaaS) for modular smart contract execution, Private Data Collections (PDC) for secure handling of sensitive information, and dynamic service discovery for efficient multi-organisation interaction. These components collectively ensure data integrity, confidentiality, and system scalability.

The web application layer of NEVS comprises a role-based registration and authentication system supporting employees, employers, and administrators. The platform incorporates a secure document upload mechanism with AES-256 encryption and Cloudinary private storage, a hybrid automated verification engine employing OCR-based document analysis and risk scoring, and a Trusted Job Portal accessible exclusively to verified users. An administrative review portal enables manual oversight of verification requests with complete audit trail support.

NEVS introduces a verifiable and tamper-resistant employment verification layer, enabling secure collaboration between government authorities, organisations, and third-party verification platforms, whilst safeguarding candidates against fraudulent practices and data misuse. The system was validated through comprehensive testing across all modules, with all critical test cases returning successful results.

TABLE OF CONTENTS

TABLE OF CONTENTS

TITLE		I
DECLARATION OF ORIGINALITY		I
ACKNOWLEDGEMENTS		II
ABSTRACT		III
TABLE OF CONTENTS		V
LIST OF FIGURES		II
LIST OF TABLES		X
LIST OF ABBREVIATIONS		XI
CHAPTER 1: INTRODUCTION		
1.1	Problem Statement and Motivation	1
1.2	Objectives	2
1.3	Project Scope and Direction	3
1.4	Report Organisation	4
CHAPTER 2: LITERATURE REVIEW		
2.1	Methodology	5
2.2	Summary of Findings	5

CHAPTER 3: SYSTEM REQUIREMENT		
3.1	Introduction	9
3.2	Software and Hardware Requirements	10
3.3	Summary	11
CHAPTER 4: SYSTEM DESIGN		
4.1	Introduction	12
4.2	Proposed System	12
4.3	Data Flow Diagram	15
4.4	Summary	15
CHAPTER 5: IMPLEMENTATION		
5.1	Introduction	16
5.2	Registration and Authentication Module	16
5.3	Document Upload and Encryption Module	19
5.4	Automated Verification Engine	20
5.5	Admin Review Portal	22
5.6	Employer Dashboard Module	24
5.7	Trusted Job Portal Module	25
5-8	Blockchain Gateway and Chaincode Architecture	26
5-9	Packages and Libraries Used	30
CHAPTER 6: SYSTEM TESTING		

6.1	Introduction	34
6.2	Test Cases	34
6.3	Results	36
6.4	Performance Evaluation	37
6.5	Summary	37
CONCLUSION		38
REFERENCES		40

Appendix	TITLE	Page
A	Registration Forms	42
B	Weekly Reports	45
C	Research Paper	102

LIST OF FIGURES

Figure Number	Title	Page
Figure 1.1	High-Level NEVS Ecosystem Overview	2
Figure 2-1	Traditional BGV vs. NEVS Blockchain Verification Process	6
Figure 4.1	Layered System Architecture	12
Figure 4.2	Fabric Network Topology Diagram	13
Figure 4.3	NEVS System Data Flow Diagram	15
Figure 5.1	Employment Lifecycle State Machine	28
Figure 5.2	CCaaS Deployment Architecture	27
Figure 5.3	End-to-End Transaction Flow with Private Data	29
Figure 5.4	Role-Based Registration Flow	17
Figure 5.5	Document Upload and Encryption Flow	20
Figure 5.6	Hybrid Automated Verification Engine Pipeline	22
Figure 5.7	Admin Review Portal Workflow	23
Figure 5.8	Employer Dashboard Module Flow	24
Figure 5.9	Trusted Job Portal Access and Application Flow	26
Figure 5-10	NEVS Home Page – Platform Overview	32
Figure 5-11	NEVS Login Interface – Role-Based Authentication	32
Figure 5-12	Employee & Employer Registration Onboarding Form	32
Figure 5-13	Employee Dashboard – Verified Identity Document	33
Figure 5-14	Employer Dashboard – Company Verification Status	33
Figure 5-15	NEVN Central Authority – Admin Document Review Queue	33

LIST OF TABLES

Table Number	Title	Page
Table 1-1	Tiered Company Participation Model	4
Table 3-1	Functional Requirements	10
Table 3-2	Non-Functional Requirements	10
Table 3-3	Software Stack — Blockchain Layer	10
Table 3-4	Software Stack — Web Application Layer	11
Table 3-5	Hardware Requirements	11
Table 4.1	Network Topology Components	13
Table 4.2	Multi-Contract Chaincode Architecture	14
Table 4.3	Private Data Collection Configuration	14
Table 5.1	Implementation Phases	16
Table 5.2	Employee Registration Database Schema	18
Table 5.3	Employer Registration Database Schema	18
Table 5.4	Authentication API Endpoints	19
Table 5-5	Document Upload API Endpoints	20
Table 5-6	Verification Risk Scoring System	22
Table 5-7	Verification Status Decision Logic	22
Table 5-8	Admin Review API Endpoints	24
Table 5-9	Employer Dashboard API Endpoints	25
Table 5-10	Job Portal API Endpoints	26
Table 5-11	Employment Lifecycle State Transitions	28
Table 5-12	Active REST API Gateway Endpoints	30
Table 5-13	Packages and Libraries — Blockchain Layer	30
Table 5-14	Packages and Libraries — Gateway and Web Application Layer	31
Table 5-15	Packages and Libraries — Frontend	31
Table 6.1	System Test Cases	34
Table 6.2	Performance Evaluation Metrics	36

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AES	Advanced Encryption Standard
API	Application Programming Interface
BGV	Background Verification
CA	Certificate Authority
CCaaS	Chaincode-as-a-Service
CRUD	Create, Read, Update, Delete
DLT	Distributed Ledger Technology
DRC	Department Review Committee
DPDP	Digital Personal Data Protection
HA	High Availability
HTTP	Hypertext Transfer Protocol
IV	Initialisation Vector
JWT	JSON Web Token
JSON	JavaScript Object Notation
KYC	Know Your Customer
LAN	Local Area Network
MSP	Membership Service Provider
mTLS	Mutual Transport Layer Security
NCA	NEVN Central Authority
NEVN	National Employment Verification Network
NEVS	National Employment Verification System
OCR	Optical Character Recognition
OBE	Outcome Based Education
PDC	Private Data Collection
PII	Personally Identifiable Information
PO	Programme Outcome
PSO	Programme Specific Outcome
REST	Representational State Transfer
SDK	Software Development Kit
SEE	Semester End Examination
SHA	Secure Hash Algorithm
SSE	Server-Sent Events

Abbreviation	Full Form
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
VPC	Virtual Private Cloud

CHAPTER 1 — INTRODUCTION

1-1 Problem Statement and Motivation

Modern hiring and employment verification remained deeply fragmented, costly, and vulnerable to systemic fraud. Industry studies indicated that employers spent over USD 2,500 per employee per year on HR functions, with traditional third-party background verification costing USD 50–100 per individual request and upwards of USD 2,800 for a comprehensive check. These processes relied on telephone-based outreach, physical document exchange, and manual cross-referencing — methods that were inherently slow, error-prone, and trivially defeatable by a motivated fraudster wielding falsified documentation.

A fundamental asymmetry existed: employers possessed employment records but shared them reluctantly, whilst employees had no standardised mechanism to assert ownership or control over their own work history. This vacuum created a market for fabricated resumes, dummy hiring schemes, and data-harvesting scams masquerading as legitimate job postings.

This project was directly motivated by such a real-world encounter. During the application process on Internshala, one of the authors observed a company collecting extensive personal data — including psychological profiling questions and physical addresses — through external forms served even to rejected candidates. Further investigation revealed a pattern: the company had posted a disproportionately high number of opportunities relative to an abnormally low count of actual hires. This raised critical questions regarding whether certain entities exploited hiring platforms to harvest student data rather than genuinely recruit, and how manipulation through fake hires via dummy accounts could be systematically prevented.

These questions motivated the design of NEVS: a system-level solution that treated each employment event as a trusted, on-chain credential from the point of creation, thereby eliminating post-hoc verification entirely and rendering fabricated work histories computationally detectable.

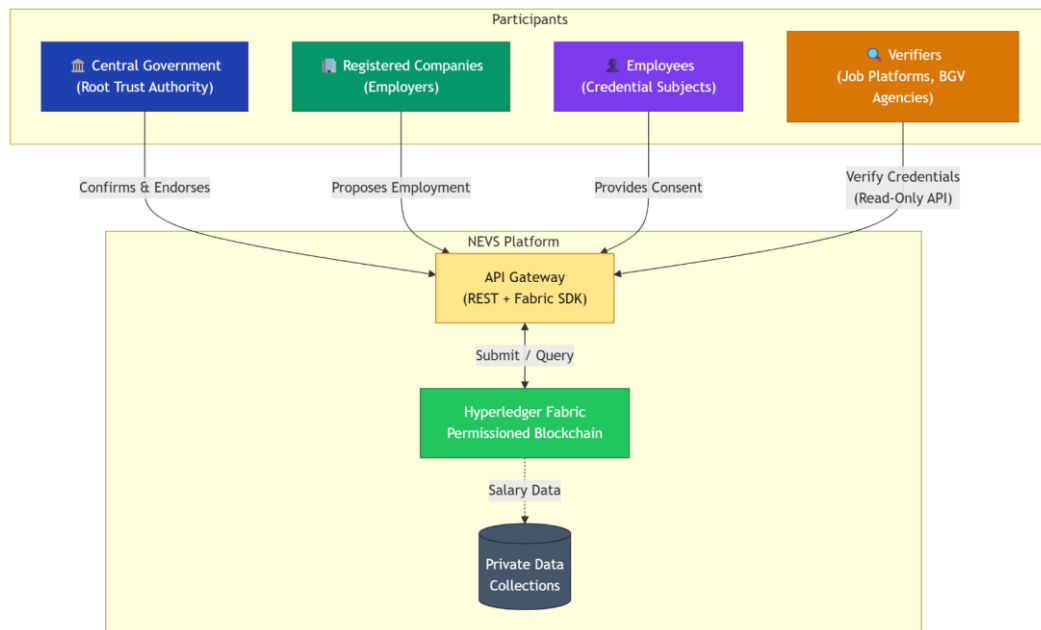


Figure 1-1 — High-Level NEVS Ecosystem Overview

1-2 Objectives

The National Employment Verification System (NEVS) was designed with the following principal objectives:

1. **Immutable Employment Records:** To record each hiring, confirmation, or termination event as a blockchain transaction on a permissioned ledger, ensuring committed data could not be retroactively altered, deleted, or disputed without cryptographic evidence.
2. **Two-Party Consent Model:** To enforce explicit digital consent from both the employer and the employee before any employment record achieved confirmed status on the ledger, satisfying legal and ethical requirements whilst preventing unilateral or fraudulent record creation.
3. **Privacy-Preserving Architecture:** To store only non-sensitive metadata on-chain whilst routing all personally identifiable information through Hyperledger Fabric Private Data Collections (PDC) with strict access policies.
4. **Fraud Detection and Elimination:** To anchor records in a permissioned ledger governed by recognised organisational authorities, making fabricated employment claims computationally infeasible.

5. Scalable, Multi-Organisation Architecture: To design the network to natively support multiple organisations operating their own infrastructure within a shared trust framework.
6. Real-Time Verification API: To provide a programmatic, read-only REST API enabling instant employment history verification without accessing underlying personal data.
7. Secure Web Application Platform: To build a comprehensive role-based web application supporting employee and employer registration, document upload with AES-256 encryption, automated verification with OCR-based document analysis, a Trusted Job Portal for verified users, and an administrative review portal with complete audit trail support.

1-3 Project Scope and Direction

What NEVS Covers

NEVS addressed the end-to-end lifecycle of formal employment records. Its scope spanned the initiation of a hiring proposal by a company, through employee consent collection, government-backed confirmation, and eventual termination — all recorded as immutable transactions on a permissioned blockchain. The system provided APIs for both write operations (restricted to authorised organisational gateways) and read operations (available to any verifier). The web application layer encompassed role-based registration, secure document management, hybrid verification processing, employer workforce management, and a trust-gated job marketplace.

What NEVS Does Not Cover

NEVS was explicitly not a job portal in the conventional sense, social networking platform, or payroll system. It did not replace platforms such as Internshala or LinkedIn; rather, it provided the authoritative data layer that such platforms could query. It did not process salary disbursements, tax calculations, or benefits administration — only the core employment credential events.

Tiered Company Participation Model

Tier	Description	Network Access
Tier 1 (Small Companies)	Interact exclusively through the Government Gateway REST API	No direct Fabric access
Tier 2 (Medium Companies)	Optionally run a read-only peer for local query performance	Read-only ledger access
Tier 3 (Large Enterprises)	Operate their own MSP, peers, and endorsement authority	Full endorsement participation

Table 1-1 Tiered Company Participation Model.

1-4 Report Organisation

The remainder of this report was organised as follows. Chapter 2 presented a structured literature review of existing employment verification solutions, blockchain credential platforms, and government pilot programmes. Chapter 3 detailed the functional and non-functional system requirements, along with the complete software and hardware stack. Chapter 4 described the system architecture, network topology, chaincode design, and data flow. Chapter 5 provided a comprehensive account of the implementation across all modules — registration, authentication, document upload, automated verification, admin review, employer dashboard, job portal, and blockchain gateway. Chapter 6 reported the testing methodology, test cases, results, and performance evaluation. The report concluded with a summary of achievements, limitations, and directions for future work.

CHAPTER 2 — LITERATURE REVIEW

2-1 Methodology

A structured survey of industry solutions, government pilot programmes, and academic research related to blockchain-based employment and credential verification was conducted. The review methodology encompassed searches on Google Scholar and IEEE Xplore using terms such as 'blockchain background verification', 'permissioned ledger employment records', and 'digital credential platform'. Industry analysis involved review of official documentation, whitepapers, and company profiles for platforms including MployChek, CSM ProofChain, the Velocity Network Foundation, and Virtualness (formerly Verix). Government pilot reports and press releases from Canada's Talent Cloud blockchain pilot, India's EDUChain initiative, and Estonia's KSI-based digital identity infrastructure were also examined. Each source was evaluated against criteria of trust model, architecture, privacy design, and practical applicability to the NEVS context.

Rationale for Hyperledger Fabric

Several blockchain platforms were considered during the design phase. Public blockchains such as Ethereum and Solana were rejected due to their open-membership model, non-deterministic finality, and gas-cost economics — all incompatible with a government-regulated employment system handling sensitive national data. Among permissioned alternatives, Hyperledger Fabric was selected for its modular architecture with pluggable consensus (Raft), endorsement policies, and private data collections; its identity-first design wherein every participant held an X.509 certificate issued by a Membership Service Provider (MSP); channel isolation enabling multiple organisations to maintain shared or private ledgers within the same network; chaincode flexibility supporting Go, Node.js, or Java contracts deployed externally as a service; and enterprise adoption backed by the Linux Foundation with production use by IBM, Walmart, and Maersk.

2-2 Summary of Findings

Traditional Background Verification vs. Blockchain Solutions

Conventional background-verification firms — HireRight, Sterling, First Advantage — operated reactively, engaged after a hiring decision and manually contacting previous

employers and government agencies. This process typically took days to weeks and cost USD 50–100 per check. The fundamental flaw was temporal: verification occurred too late, after candidates may have already been onboarded with falsified credentials.

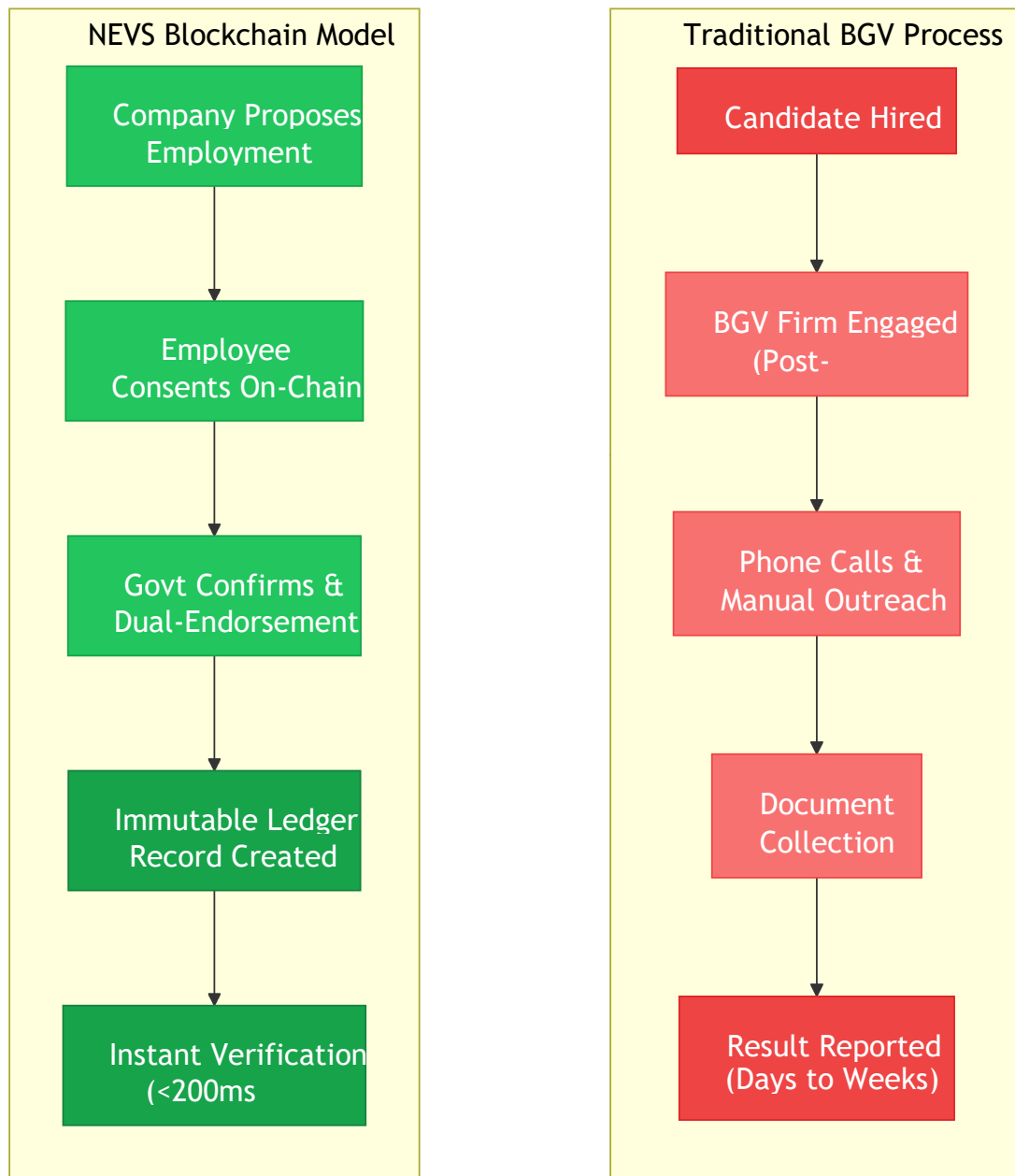


Figure 2-1 — Traditional BGV vs. NEVS Blockchain Verification Process

Figure 2-1 illustrates the contrast between the two verification approaches. The NEVS blockchain model — Company Proposes, Employee Consents On-Chain, Government Confirms with Dual-Endorsement, Immutable Ledger Record Created, Instant

Verification — inverted the traditional post-hire process entirely, achieving verification at source rather than retrospectively.

Emerging blockchain platforms such as MployChek (India-based, document hashing for background verification) and CSM ProofChain (decentralised, tamper-proof employment data) demonstrated the viability of the approach but remained vendor-specific and lacked government backing.

Decentralised Credential Networks

The Velocity Network Foundation supported by SAP, Oracle, ZipRecruiter, and others constructed a vendor-neutral, self-sovereign credential ledger. Its premise was that verifying career records could take days, weeks, or even months; the mainnet aimed to enable employers to verify a candidate's diplomas, certifications, and work experience almost instantaneously. Workers maintained credentials in a blockchain-backed wallet and shared them via public keys. Virtualness (formerly Verix) pursued a similar vision using NFT-based certificates for educational and professional credentials.

Government and Academic Pilots

Several governments explored blockchain for credential infrastructure. Canada's government created blockchain-based 'Blockcerts' for project-based workers, providing secure digital credentials for skills and experience. In India, the EDUChain initiative a collaboration involving Open Campus and the Government of Madhya Pradesh digitised 50 million student and graduate academic records on blockchain, enabling instant verification of educational credentials and cutting administrative overhead for employers. Estonia's KSI blockchain underpinned its national digital identity system, providing tamper-proof records for government services.

Case Study — Saudi Aramco

Saudi Aramco developed a Blockchain Certificate Verifier for heavy-equipment operator credentials. By storing cryptographic hashes of training certificates on-chain, verification time was reduced from approximately two weeks to under five seconds. This enterprise deployment validated the practical power of blockchain verification in regulated industrial contexts.

Academic Proposals DocsLedger

DocsLedger, a permissioned blockchain concept studied in academic literature, proposed a system where employers hashed employment documents on-chain at creation and employees carried verified records in a digital wallet to share with third-party verifiers. The architecture emphasised permissioned access, off-chain storage for privacy, and cryptographic signatures principles directly adopted in NEVS.

Conclusion of Literature Review

A clear trend emerged from the review: the industry was shifting from retroactive verification to credential issuance at source. However, most existing platforms were either vendor-specific or consortium-led, lacking the government-backed authority necessary for national-scale legal enforceability. NEVS was distinct in positioning the government as the root trust authority whilst enforcing dual-party consent, combining the cryptographic guarantees of blockchain with the legal weight of state endorsement.

CHAPTER 3 — SYSTEM REQUIREMENTS

3-1 Introduction

NEVS required a carefully engineered combination of blockchain infrastructure, web application middleware, and security controls. Requirements were categorised as functional (defining system behaviour) and non-functional (defining system qualities). The complete platform comprised two primary layers: the Hyperledger Fabric permissioned blockchain network and the NEVS web application platform.

Functional Requirements

Requirement	Description
Role-Based Registration	Employees and employers registered through a government-portal-style interface with role-specific fields. Accounts were initialised with UNVERIFIED status.
Secure Authentication	JWT-based authentication was implemented for employees, employers, and administrators with bcrypt password hashing and role-specific login endpoints.
Document Upload	Employees uploaded identity documents with AES-256 encryption before Cloudinary private storage. Signed URLs provided time-limited access.
Automated Verification	A hybrid verification engine evaluated documents through format validation, OCR extraction, simulated database checks, and risk scoring to determine verification status.
Admin Review Portal	Administrators reviewed UNDER_REVIEW documents with capabilities to approve, reject, or request re-upload, with full audit trail logging.
Record Employment	Authorised organisations submitted employment events (hire, confirm, terminate) with metadata to the blockchain ledger.
Employee Consent	Every employment proposal required explicit employee approval before ledger commitment.
Immutable Ledger Storage	Consented employment events were appended to a permissioned blockchain with cryptographic finality and dual-organisation endorsement.
Query and Verification	A public REST API enabled read-only queries against verified employment history with response latency below 200 milliseconds.
Trusted Job Portal	A job marketplace accessible exclusively to users with VERIFIED account status.
Employer Dashboard	Employers managed company verification, employee records, and verification request workflows through a dedicated dashboard.

Requirement	Description
Privacy Controls	Sensitive data was isolated via Private Data Collections or stored off-chain with strict access controls.

Table 3-1 Functional Requirements

Non-Functional Requirements

Requirement	Description
Security	X.509 certificate-based identity, mTLS encryption, AES-256 document encryption, bcrypt password hashing, JWT session management, and transient data routing for PDC.
Performance	Sub-second block commitment under moderate load; API query latency below 200 milliseconds; document upload processing within acceptable user wait times.
Scalability	Network architecture supported adding new organisations without downtime; stateless REST API horizontally scalable behind a load balancer.
Reliability	Raft-based ordering service with distributed peer topology; automated recovery scripts enabling complete network restoration in under 60 seconds.
Compliance	Architecture designed to comply with India's Digital Personal Data Protection (DPDP) Act; sensitive data isolation through PDC and off-chain storage.
Usability	RESTful API with clear endpoint semantics; React-based web dashboard with role-appropriate navigation and verification status indicators.

Table 3-2 Non-Functional Requirements

3-2 Software and Hardware Requirements

Software Stack — Blockchain Layer

Component	Technology	Version	Purpose
Blockchain Platform	Hyperledger Fabric	2.5	Permissioned DLT core
Certificate Authority	Hyperledger Fabric CA	1.5.7	X.509 identity issuance per MSP
Smart Contracts	Go (Golang)	1.18	Chaincode implementing employment logic
Chaincode Framework	fabric-contract-api-go	1.2.2	Contract API for structured chaincode
Chaincode Infrastructure	fabric-chaincode-go (shim)	—	CCaaS server implementation
Containerisation	Docker + Docker Compose	24.x / 3.7	Network component orchestration
State Database	LevelDB	Built-in	Peer key-value world state storage

Table 3-3 Software Stack — Blockchain Layer

Software Stack — Web Application Layer

Component	Technology	Version	Purpose
API Gateway	Node.js + Express	20.x / 4.19.2	REST middleware between clients and Fabric
Fabric SDK	fabric-network	2.2.20	Gateway-to-peer transaction submission
Database	PostgreSQL (Docker)	15.x	Off-chain user data, documents, and verification records
File Storage	Cloudinary (Private)	—	Encrypted document storage with signed URL access
OCR Engine	Tesseract.js	5.x	Document text extraction for verification
Authentication	JWT + bcrypt	—	Session management and password hashing
Frontend Framework	React.js + TypeScript + Vite	18.x	User interface for all roles
CSS Framework	Tailwind CSS / Vanilla CSS	—	Responsive UI styling
Animation Library	Framer Motion	—	UI transition animations

Table 3-4 Software Stack — Web Application Layer

Hardware Requirements

Component	Minimum Specification
Development Machine	8 GB RAM, 4-core CPU, 50 GB SSD, Docker Desktop
Production Deployment	Ubuntu Server 22.04 LTS or equivalent, 16 GB RAM per peer node
Optional Distributed Deployment	Raspberry Pi 4 (8 GB) for LAN-based peer hosting
Network	Gigabit Ethernet (LAN) or cloud VPC with low-latency inter-node connectivity

Table 3-5 Hardware Requirements

3-3 Summary

NEVS required a dual-layer technology stack: a permissioned blockchain infrastructure comprising Hyperledger Fabric peers, orderers, and Certificate Authorities; and a web application platform comprising a Node.js API Gateway, PostgreSQL database, Cloudinary private document storage, Tesseract.js OCR engine, and React-based frontend. The architecture was intentionally modular, enabling each layer to be developed, tested, and upgraded independently.

CHAPTER 4 — SYSTEM DESIGN

4-1 Introduction

NEVS was designed as a layered, multi-organisation architecture where the core permissioned blockchain ledger stored all finalised employment transactions. Surrounding this core was a government-controlled API Gateway enforcing policy, identity, consensus, and consent before any state change occurred. The foundational design principle was authority separation combined with decentralised trust: no single organisation could unilaterally write to the ledger.

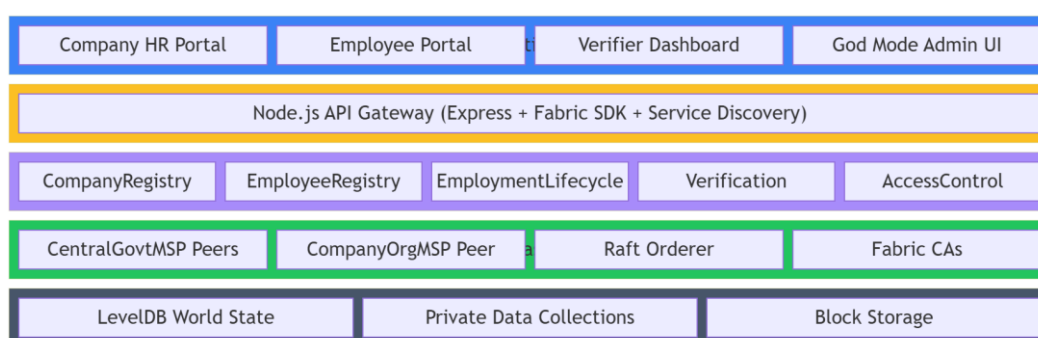


Figure 4-1 — Layered System Architecture

Figure 4-1 illustrates the five-layer system architecture: User Interface (Company HR Portal, Employee Portal, Verifier Dashboard, God Mode Admin UI), API Gateway (Node.js Express with Fabric SDK and Service Discovery), Smart Contract layer (CompanyRegistry, EmployeeRegistry, EmploymentLifecycle, Verification, AccessControl), Fabric Network (CentralGovtMSP Peers, CompanyOrgMSP Peer, Raft Orderer, Fabric CAs), and Data Storage (LevelDB World State, Private Data Collections, Block Storage).

4-2 Proposed System

Network Topology

Component	Identity	Address	Role
Raft Orderer	OrdererMSP	orderer.nevs.gov:7050	Transaction ordering and block creation
Govt Peer 0	CentralGovtMSP	peer0.central.govt:7051	Primary endorser and anchor peer (Government)

Component	Identity	Address	Role
Govt Peer 1	CentralGovtMSP	peer1.central.govt:8051	Secondary endorser and HA failover
Company Peer 0	CompanyOrgMSP	peer0.company.org:9051	Endorser and anchor peer (Company Org)
Orderer CA	—	ca.nevs.gov:8054	Orderer identity management
Govt CA	—	ca.central.govt:7054	Government identity management
Company CA	—	ca.company.org:9054	Company Org identity management
Employment CCaaS	—	employment.cc:9999	External chaincode execution container
API Gateway	—	localhost:3000	REST interface with Fabric SDK

Table 4-1 Network Topology Components

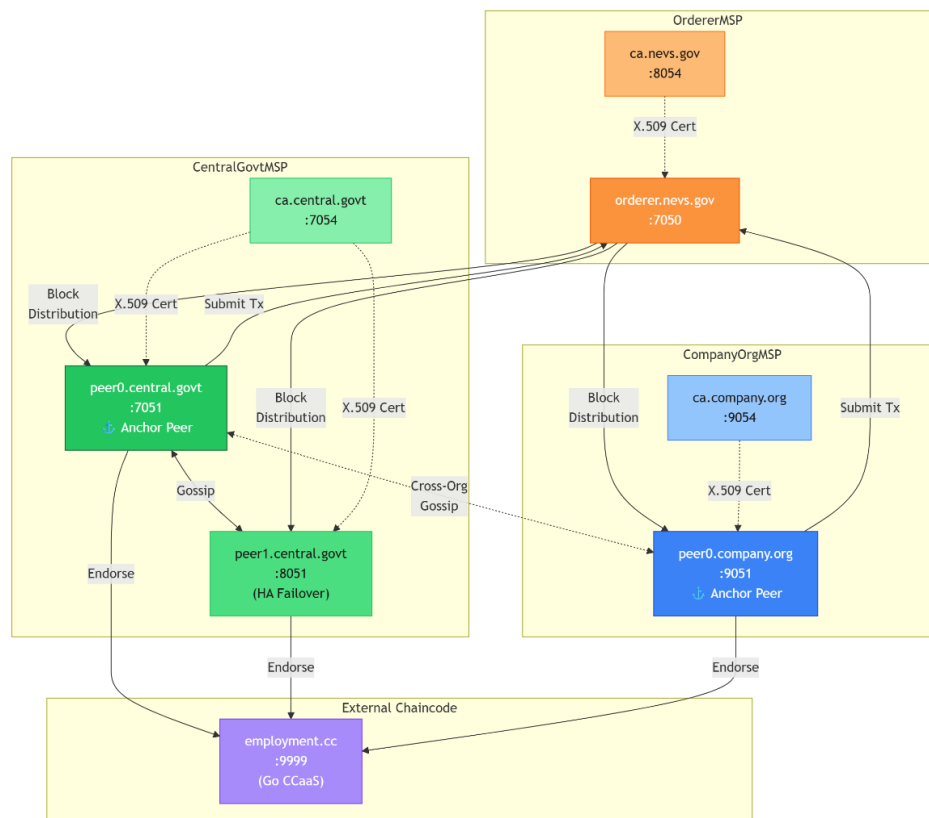


Figure 4-2 — Fabric Network Topology Diagram

Multi-Contract Chaincode Architecture

Contract	Domain	Key Functions
CompanyRegistryContract	Company onboarding and lifecycle	RegisterCompany, ApproveCompany, SuspendCompany

Contract	Domain	Key Functions
EmployeeRegistryContract	Employee identity management	RegisterEmployee, GetEmployee, UpdateEmployee
EmploymentLifecycleContract	Employment state machine	ProposeEmployment, EmployeeConsent, ConfirmEmployment, TerminateEmployment
VerificationContract	Third-party verification	VerifyEmployment, GenerateVerificationProof
AccessControlContract	Identity introspection	GetCallerIdentity, ValidateAccess

Table 4-2 Multi-Contract Chaincode Architecture

Endorsement Policy

The chaincode enforced a strict dual-endorsement policy: `AND('CentralGovtMSP.member', 'CompanyOrgMSP.member')`. Every write transaction required independent endorsement from at least one peer from each organisation.

Private Data Collection Configuration

Sensitive employment attributes specifically salary and compensation data — were handled through Fabric's Private Data Collections mechanism. Table 4-3 below presents the PDC configuration.

Private data was transmitted as transient data in the transaction proposal, routed directly from the client to endorsing peers over TLS. It never appeared in the transaction payload, the ordered block, or the public world state. This ensured that the orderer and any future non-member peers could not access sensitive compensation information

Parameter	Value
Collection Name	employmentPrivateData
Policy	<code>OR('CentralGovtMSP.member', 'CompanyOrgMSP.member')</code>
Required Peer Count	1
Max Peer Count	3
Block to Live	0 (indefinite retention)
Member Only Read	true
Member Only Write	true

Table 4-3 Private Data Collection Configuration

4-3 Data Flow Diagram

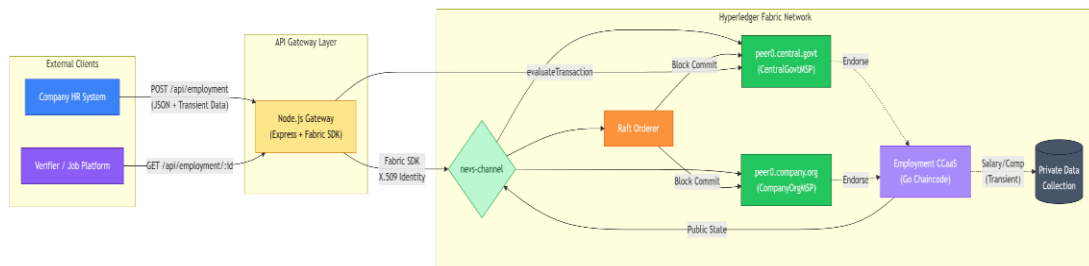


Figure 4-3 — NEVS System Data Flow Diagram

Figure 4-3 illustrates the NEVS data flow. The Write Path: Company HR Client → POST /api/employment/ → API Gateway (extract salary and compensation as transient data) → Transaction Proposal (args and transient data over TLS) → Dual-org peer endorsement (CentralGovtMSP and CompanyOrgMSP) → CCaaS invocation → PutState (public record) and PutPrivateData (salary/compensation) → Collect endorsements → Submit to Raft Orderer → Block commit → 201 Created response. The Read Path: Verifier → GET /api/employment/:id → API Gateway → evaluateTransaction on single peer → JSON response.

4-4 Summary

The proposed design achieved clean separation of concerns: cryptographic trust and consensus resided in the Fabric network and its endorsement policies; business logic and workflow orchestration resided in the chaincode contracts; and external accessibility resided in the Gateway and REST API layer. The dual-endorsement policy ensured that no single organisation could manipulate records. The PDC mechanism ensured salary and compensation data remained invisible to unauthorised participants. The web application layer, built upon this blockchain foundation, provided role-based access, secure document management, automated verification processing, and a trusted employment marketplace. This architecture fulfilled the stated objectives: immutable employment credentials, dual consent, privacy by design, and instant verifiability under a government-backed, multi-organisational trust model.

CHAPTER 5 — IMPLEMENTATION

5-1 Introduction

The NEVS system was implemented through three distinct phases, evolving from a minimal single-organisation prototype to a fully operational multi-organisation enterprise network with advanced privacy features and a comprehensive web application platform.

Phase	Focus	Key Deliverables
Phase 1	Foundation	Single-org Fabric network (CentralGovtMSP), basic Go chaincode, Docker containerisation, PostgreSQL database setup, role-based registration and authentication.
Phase 2	Gateway Interoperability	Node.js/Express REST API Gateway, fabric-network SDK integration, multi-contract chaincode, CCaaS deployment, document upload module, automated verification engine.
Phase 3	Enterprise Architecture	CompanyOrgMSP integration, dual-endorsement policy, Private Data Collections, dynamic service discovery, admin review portal, employer dashboard, trusted job portal, automated recovery scripts.

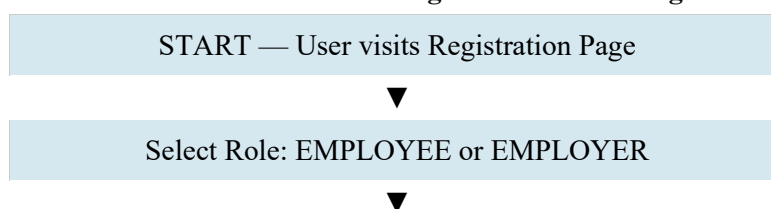
Table 5-1 Implementation Phases

5-2 Registration and Authentication Module

Frontend Registration Flow

The registration system was implemented as a government-portal-style multi-step flow with a single entry point providing role selection. Upon role selection, a common form collected email address, mobile number, password, confirm password, city, and state. Role-specific fields were subsequently presented. A Terms and Privacy Policy consent checkbox was required before final submission. Successful registration initialised accounts with UNVERIFIED status and redirected users to their role-specific dashboards.

Registration Flow Diagram



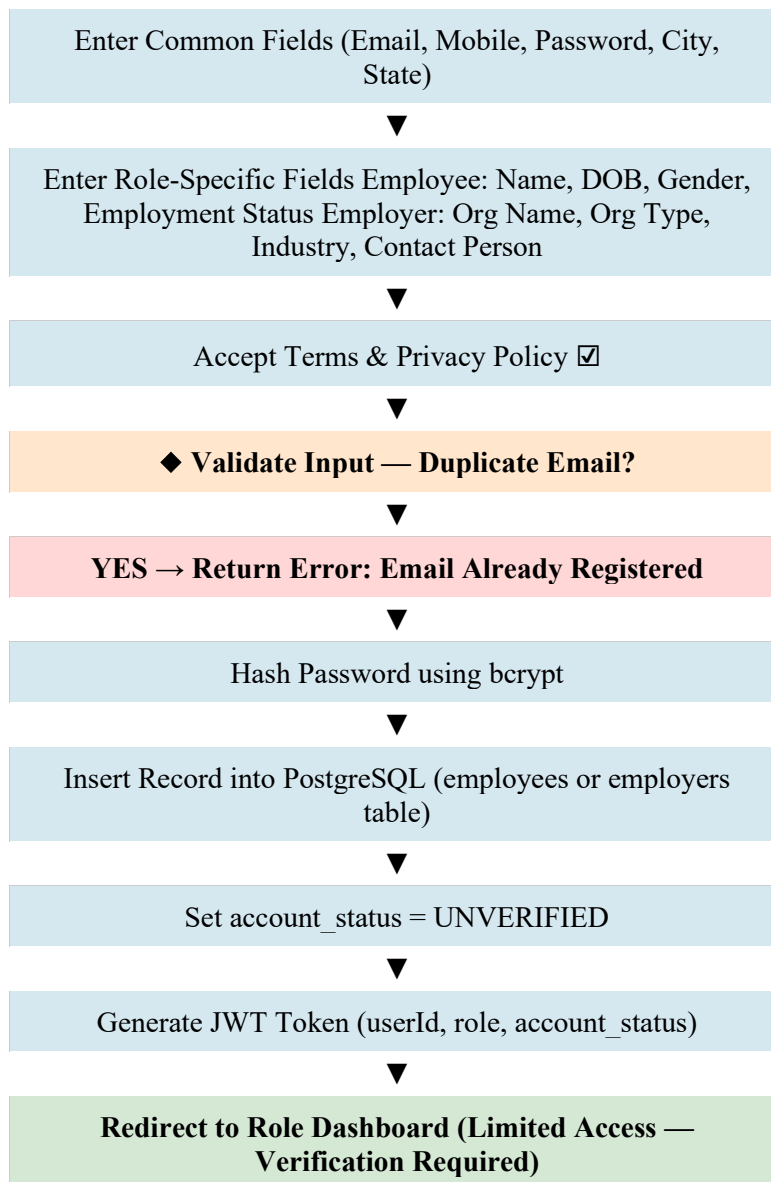


Figure 5-4 — Role-Based Registration Flow

Backend Registration — Database Schema

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY, AUTO INCREMENT	Unique employee identifier
email	VARCHAR	UNIQUE, NOT NULL	Email address for login
mobile	VARCHAR	UNIQUE, NOT NULL	Mobile number
password	VARCHAR	NOT NULL	bcrypt hashed password
full_name	VARCHAR	NOT NULL	Employee full name
date_of_birth	DATE	—	Date of birth
employment_status	VARCHAR	—	Current employment status

Column	Type	Constraints	Description
gender	VARCHAR	—	Gender (optional)
city	VARCHAR	—	City of residence
state	VARCHAR	—	State of residence
account_status	VARCHAR	DEFAULT 'UNVERIFIED'	Verification status of account
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

Table 5-2 Employee Registration Database Schema

Employer Database

Column	Type	Constraints	Description
id	SERIAL	PRIMARY KEY, AUTO INCREMENT	Unique employer identifier
email	VARCHAR	UNIQUE, NOT NULL	Email address for login
mobile	VARCHAR	UNIQUE, NOT NULL	Mobile number
password	VARCHAR	NOT NULL	bcrypt hashed password
organization_name	VARCHAR	NOT NULL	Company/organization name
org_type	VARCHAR	—	Organization type (Private/Govt/PSU)
industry_sector	VARCHAR	—	Industry sector
authorized_person_name	VARCHAR	—	Authorized person name
designation			

Table 5-3 Employer Registration Database Schema

Authentication — Login and JWT

A single login endpoint, POST /api/auth/login, supported role-based authentication. The logic searched the employees table first, then the employers table. Upon locating the user, bcrypt compared the password. If correct, a JWT token was generated containing userId, role, and account_status with a one-hour expiry. Passwords were never returned in any response.

Endpoint	Method	Role	Description
POST /api/auth/register/employee	POST	Public	Register new employee account
POST /api/auth/register/employer	POST	Public	Register new employer account
POST /api/auth/login	POST	Public	Authenticate user, return JWT token

Endpoint	Method	Role	Description
GET /api/auth/me	GET	Protected	Return current user details from JWT
POST /api/admin/login	POST	Admin	Admin-specific login, verify ADMIN role

Table 5-4 Authentication API Endpoints

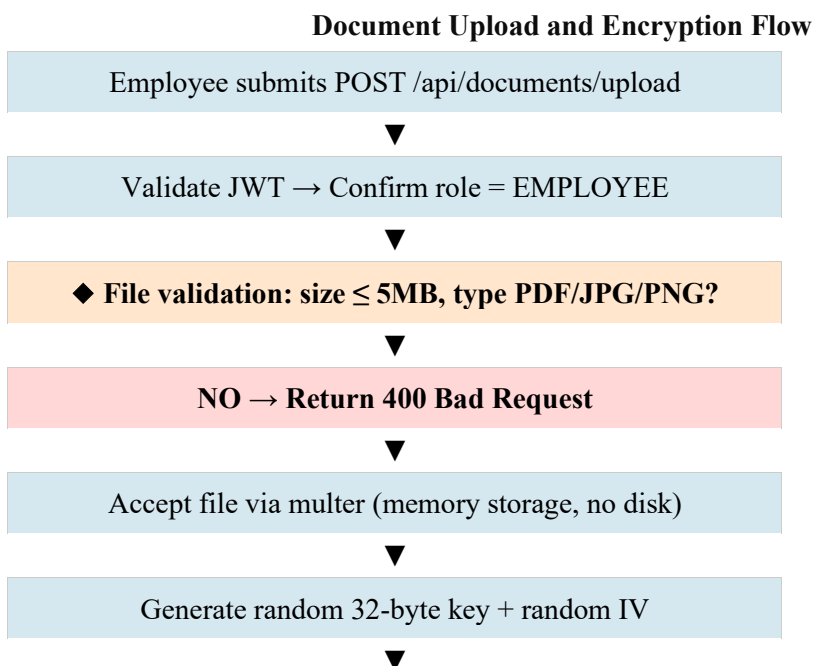
Admin Authentication

Administrator accounts were not publicly registerable. Admin accounts were created exclusively through a manual script assigning role = ADMIN with bcrypt-hashed passwords. Admin middleware protected all /api/admin/* routes by verifying the JWT and confirming the ADMIN role, returning HTTP 403 for any unauthorised access attempt.

5-3 Document Upload and Encryption Module

File Encryption Design

Before any document was uploaded to Cloudinary, it was encrypted using AES-256-CBC via the Node.js crypto module. A random 32-byte key and random Initialisation Vector (IV) were generated per file. The encryption key was itself encrypted using a master key stored in environment variables before database storage. The master encryption key was never committed to source control.



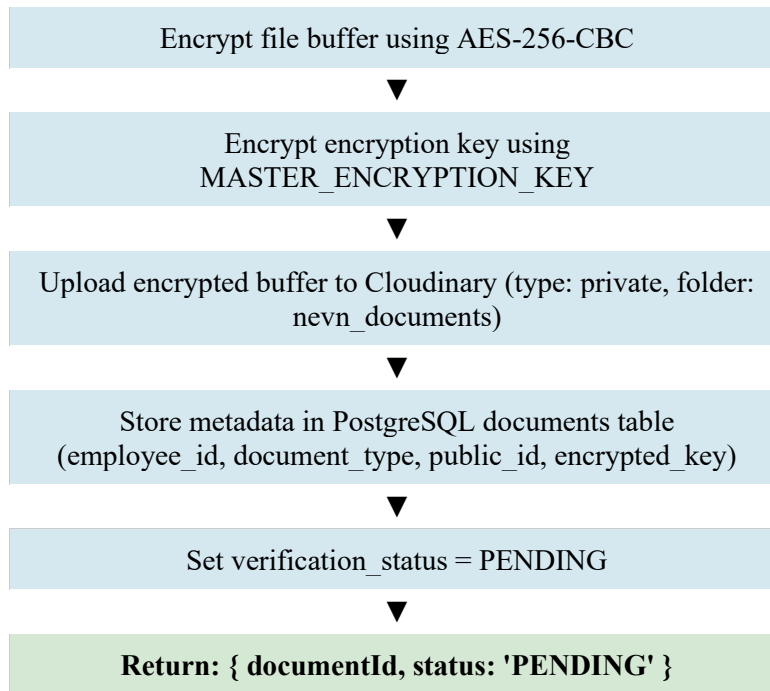


Figure 5-5 — Document Upload and Encryption Flow

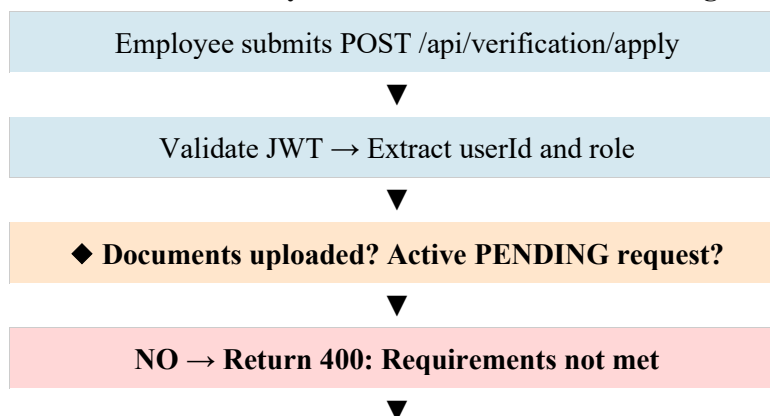
Signed URL Access

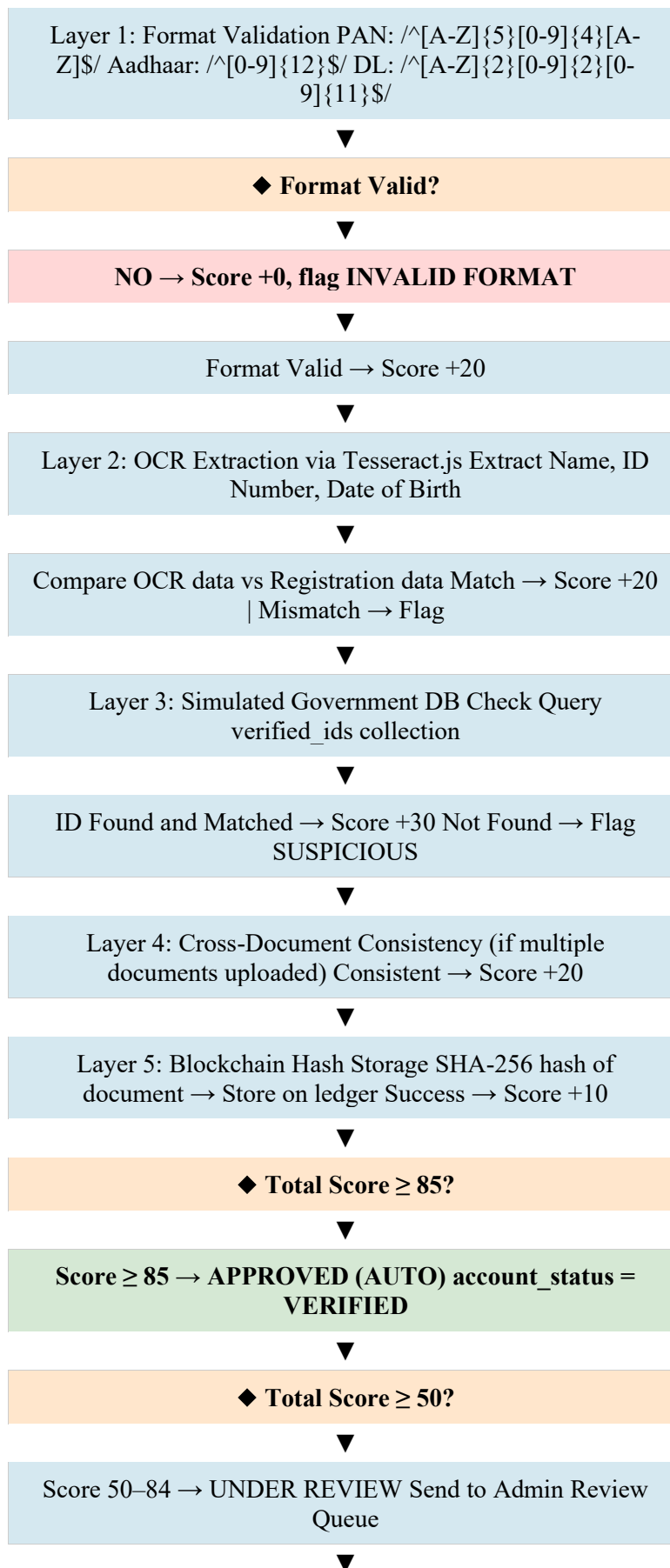
Endpoint	Method	Access	Description
POST /api/documents/upload	POST	EMPLOYEE only	Encrypt and upload document to Cloudinary
GET /api/documents/:id/view	GET	Owner or ADMIN	Generate 300-second signed URL for document access
GET /api/documents/my-documents	GET	EMPLOYEE only	List all documents for authenticated employee

Table 5-5 Document Upload API Endpoints

5-4 Automated Verification Engine

Hybrid Automated Verification Engine Pipeline





Score < 50 → REJECTED Re-application permitted

Figure 5-6 — Hybrid Automated Verification Engine Pipeline

Risk Scoring System

Verification Layer	Score Contribution	Condition
Format Valid	+20	ID format matched document-type regular expression
OCR Match	+20	Extracted name and ID matched registration data
Database Match	+30	ID found and matched in verified_ids collection
Cross-Document Consistency	+20	Name and DOB consistent across multiple documents
Blockchain Hash Stored	+10	Document hash successfully committed to ledger
Total Maximum Score	100	—

Table 5-6 Verification Risk Scoring System

Verification Status Decision Logic

Score Range	Status	Action
Score $\geq$ 85	APPROVED (AUTO)	account_status = VERIFIED, verification_level = AUTO
$50 \leq$ Score < 85	UNDER REVIEW	account_status = PENDING, sent to admin review queue
Score < 50	REJECTED	account_status = REJECTED, re-application permitted

Table 5-7 Verification Status Decision Logic

Re-Application Workflow

Re-application was permitted only where the previous verification status was REJECTED. The attempt_number was incremented, a new verification_request record was created, and historical records were preserved without deletion.

5-5 Admin Review Portal

Admin Review Portal Workflow

Admin logs in via POST /api/admin/login



Verify JWT → Confirm role = ADMIN → HTTP 403 if not

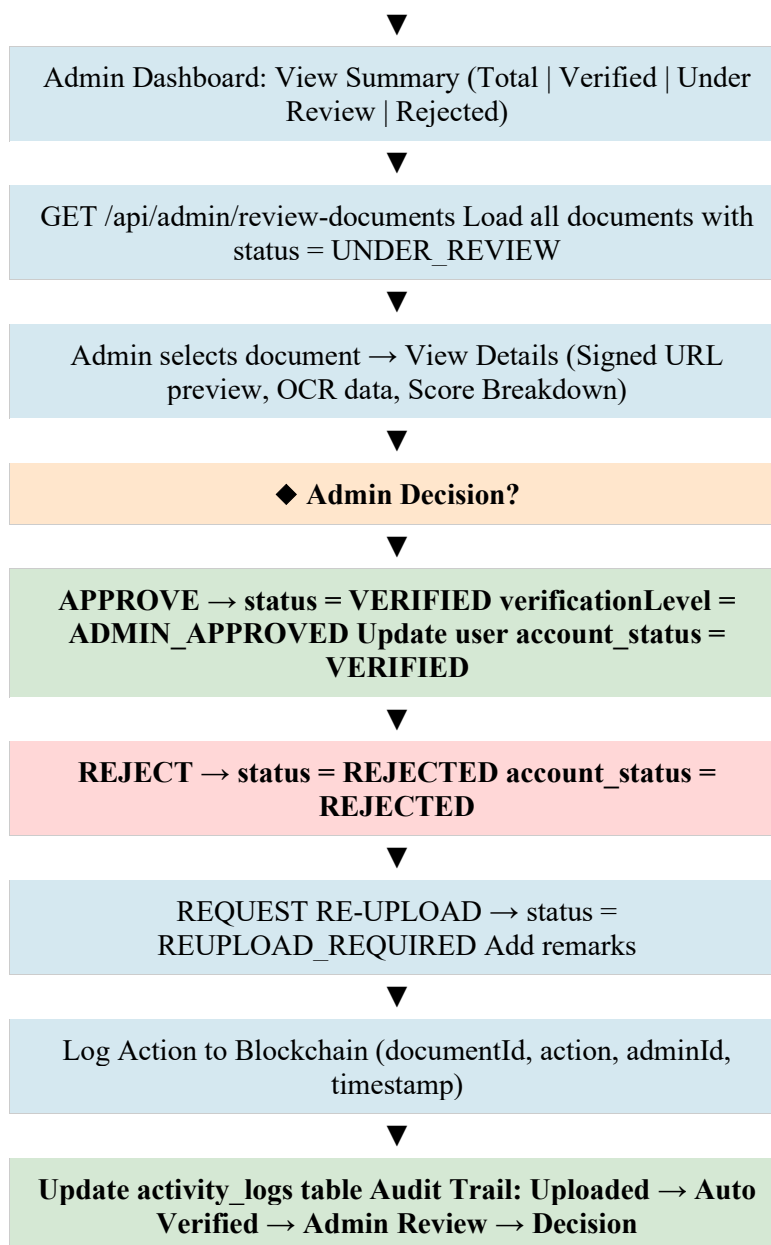


Figure 5-7 — Admin Review Portal Workflow

Endpoint	Method	Description
GET /api/admin/review-documents	GET	Retrieve all documents with status UNDER_REVIEW
GET /api/admin/document/:id	GET	Retrieve full details of a specific document for review
POST /api/admin/action	POST	Submit admin decision (approve, reject, or request re-upload)
GET /api/admin/verifications	GET	Retrieve all pending verification requests
POST /api/admin/verification/:id/approve	POST	Approve a specific verification request

Endpoint	Method	Description
POST /api/admin/verification/:id/reject	POST	Reject a specific verification request

Table 5-8 Admin Review API Endpoints

5-6 Employer Dashboard Module

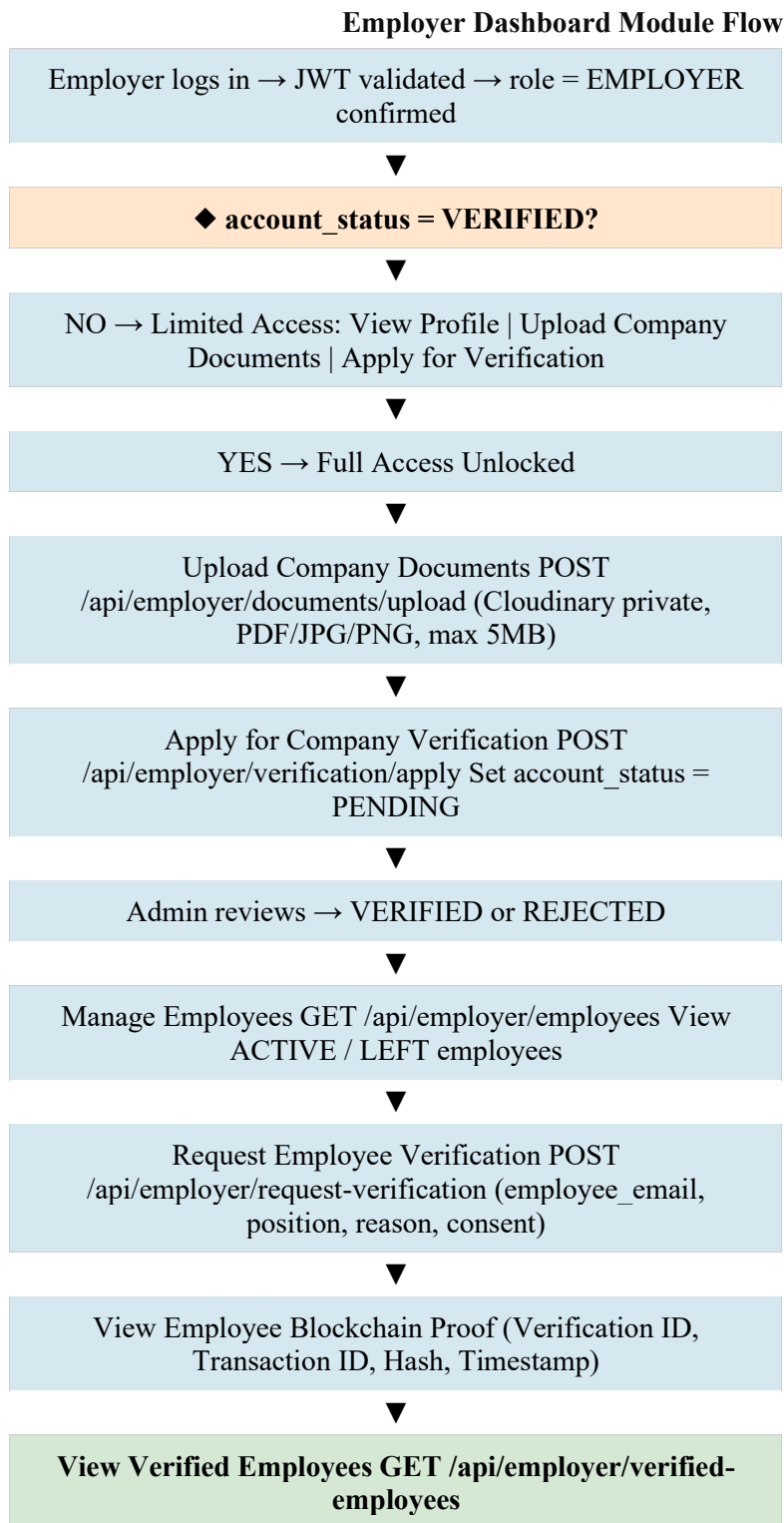


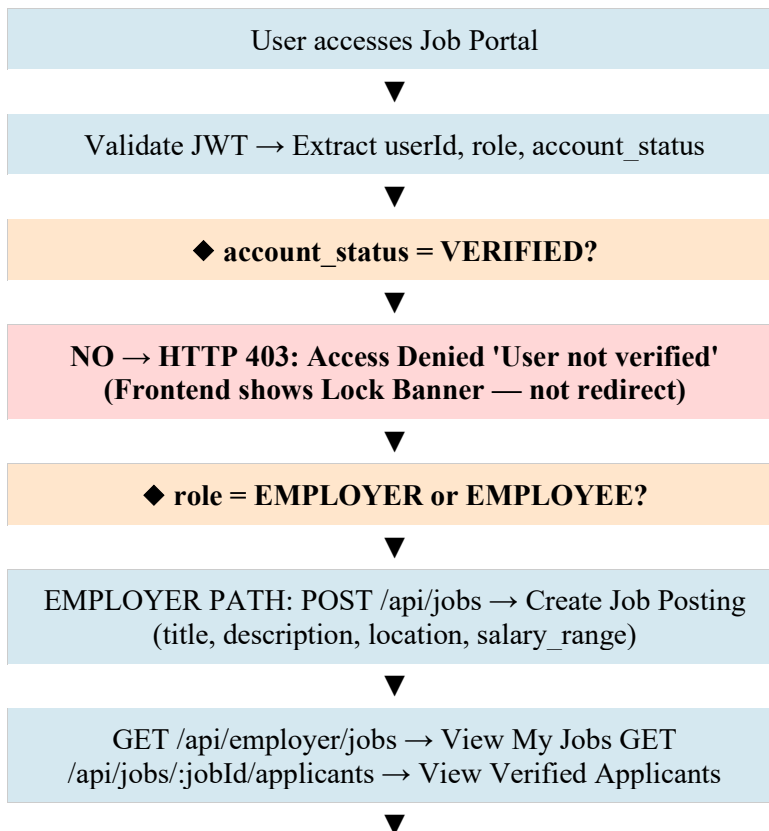
Figure 5-8 — Employer Dashboard Module Flow

Endpoint	Method	Description
GET /api/employer/profile	GET	Retrieve employer profile and account status
POST /api/employer/documents/upload	POST	Upload company verification document
POST /api/employer/verification/apply	POST	Submit company verification request
GET /api/employer/verification/status	GET	Retrieve current verification status
POST /api/employer/request-verification	POST	Request employee background verification
GET /api/employer/verification-requests	GET	Retrieve all employee verification requests
GET /api/employer/verified-employees	GET	Retrieve list of verified employees
GET /api/employer/employees	GET	Retrieve all company employees
POST /api/employer/employees/:id/leave	POST	Mark an employee as having left the company

Table 5-9 Employer Dashboard API Endpoints

5-7 Trusted Job Portal Module

Trusted Job Portal Access and Application Flow



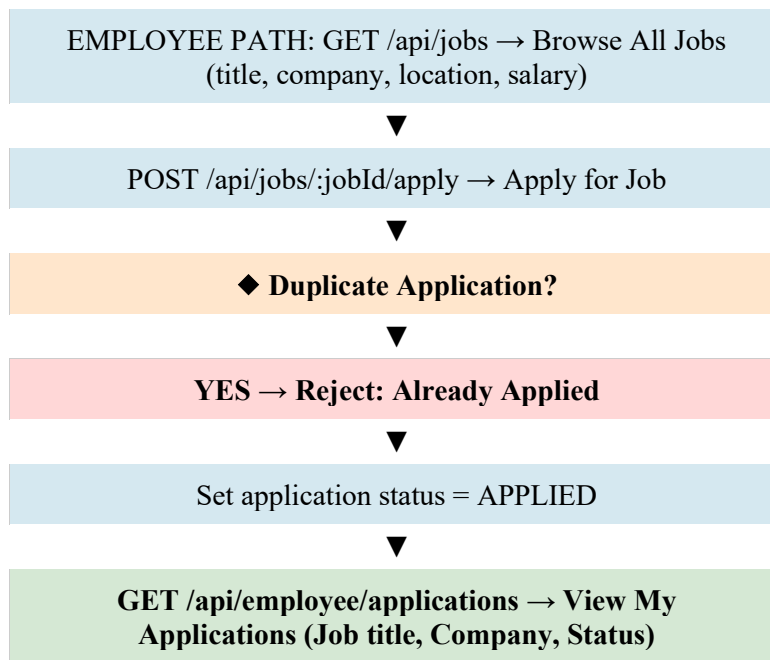


Figure 5-9 — Trusted Job Portal Access and Application Flow

Endpoint	Method	Role	Description
POST /api/jobs	POST	EMPLOYER (VERIFIED)	Create a new job posting
GET /api/jobs	GET	EMPLOYEE (VERIFIED)	Retrieve all available job listings
POST /api/jobs/:jobId/apply	POST	EMPLOYEE (VERIFIED)	Apply for a specific job posting
GET /api/employer/jobs	GET	EMPLOYER (VERIFIED)	Retrieve jobs posted by current employer
GET /api/jobs/:jobId/applicants	GET	EMPLOYER (VERIFIED)	Retrieve applicants for a specific job
GET /api/employee/applications	GET	EMPLOYEE (VERIFIED)	Retrieve applications submitted by employee

Table 5-10 Job Portal API Endpoints

5-8 Blockchain Gateway and Chaincode Architecture

Chaincode-as-a-Service (CCaaS)

Unlike the traditional model where peers built and managed chaincode containers internally, NEVS deployed chaincode as an independently managed Docker container (employment.cc) listening on port 9999. Peers connected to this external service using a connection.json metadata file embedded in the chaincode package. This architecture provided instant iteration — chaincode could be rebuilt and restarted without touching peers or channel configuration — shared execution from a single CCaaS container

servicing endorsement requests from all peers across both organisations, and process isolation with controlled resource allocation. Figure 5-2 illustrates the CCaaS deployment architecture

Gateway API Architecture

The Node.js Gateway (nevs-gateway) served as the sole external interface to the Fabric network. It connected using an X.509 identity (Admin@central.govt) and the fabric-network SDK with Service Discovery enabled. The Gateway dynamically discovered endorser layouts based on the channel's endorsement policy and routed proposals accordingly. Table 5-12 summarises the active REST API endpoints.

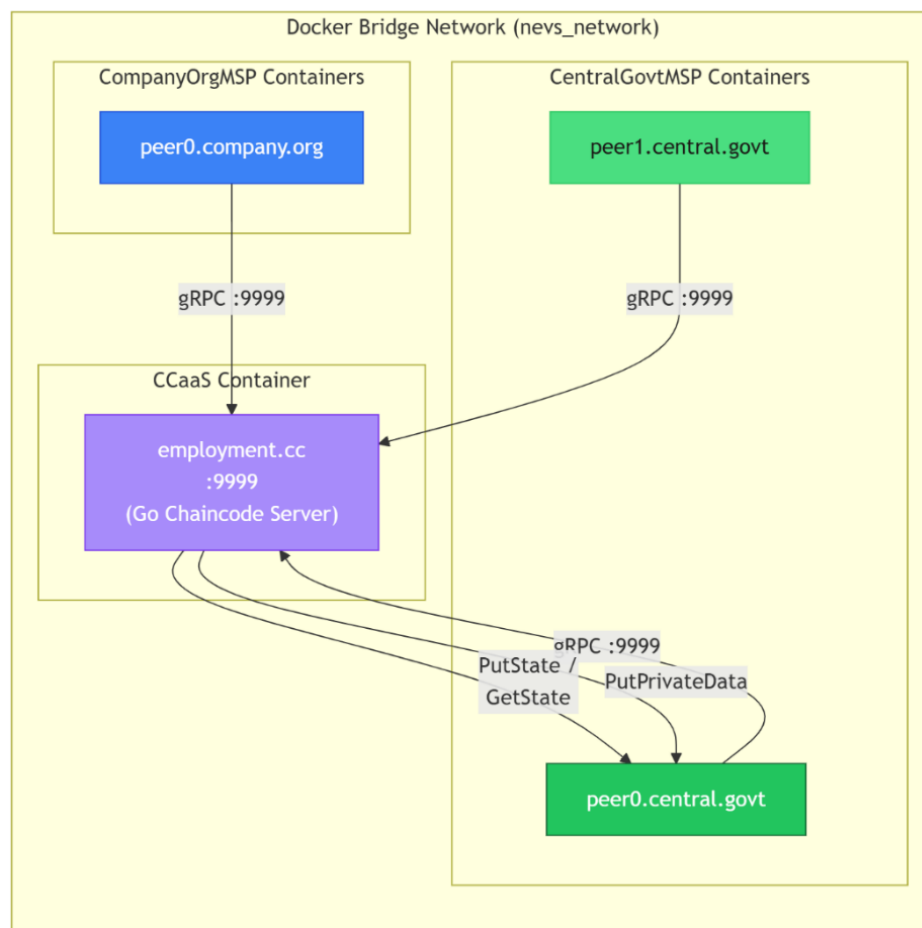


Figure 5-2 — CCaaS Deployment Architecture

Employment Lifecycle State Machine

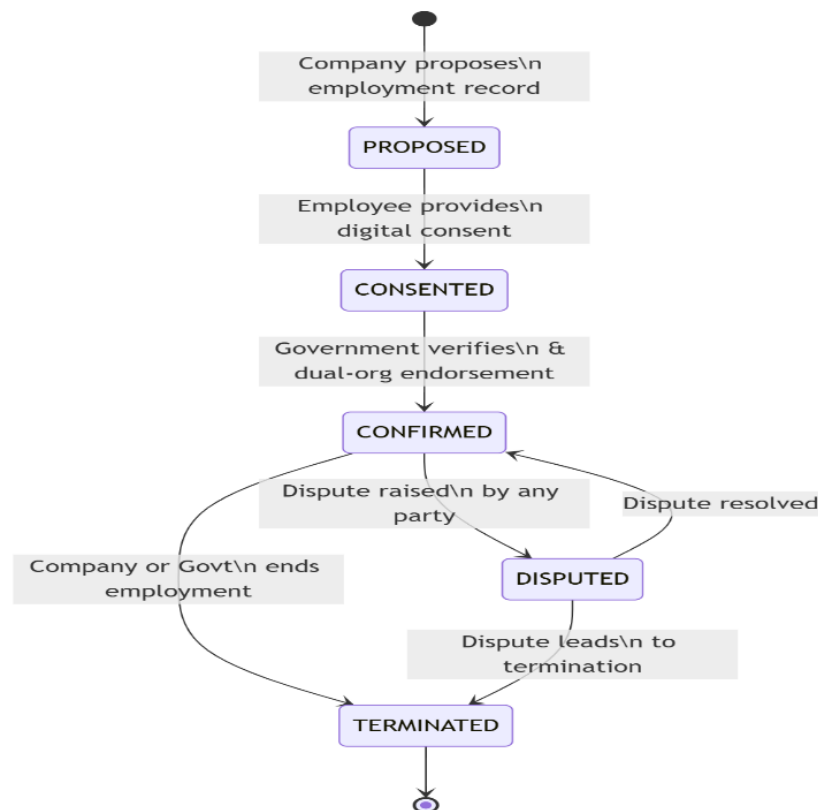


Figure 5-1 — Employment Lifecycle State Machine

Transition	Required Caller	Precondition	Action
→ PROPOSED	Company or Govt	Company APPROVED; Employee ACTIVE	Creates record with VerificationHash = SHA256(employeeId+companyId+startDate)
→ CONSENTED	Employee or Govt	Current status = PROPOSED	Records consent timestamp
→ CONFIRMED	Govt only	Current status = CONSENTED	Records confirmation timestamp; employment now active
→ TERMINATED	Company or Govt	Current status = CONFIRMED	Records end date and termination timestamp

Table 5-11 Employment Lifecycle State Transitions

End-to-End Transaction Flow with Private Data

Transient Data Handling

When the Gateway received a POST /api/employment/ request containing salary and compensation fields, it extracted these fields from the JSON body, encoded them as a Buffer, and attached them to the transaction proposal's transient field. The remaining public fields were passed as regular chaincode arguments. This ensured that sensitive compensation data travelled directly from the Gateway to endorsing peers over TLS, bypassing block inclusion and orderer visibility entirely. The chaincode's CreateEmploymentRecord function read transient data from ctx.GetStub().GetTransient() and wrote it to the employmentPrivateData collection using PutPrivateData. Figure 5-3 illustrates the end-to-end transaction flow with private data.

Automated Recovery

The restore_full_network.sh script provided one-command recovery of the complete dual-organisation network, automating Docker container startup for all peers, orderers, and CAs; channel binding and anchor peer configuration; CCaaS package installation and lifecycle approval across both organisations; and dual-endorsement policy commitment at the correct sequence number. Complete network restoration from teardown to operational status was achieved in approximately 45 seconds.

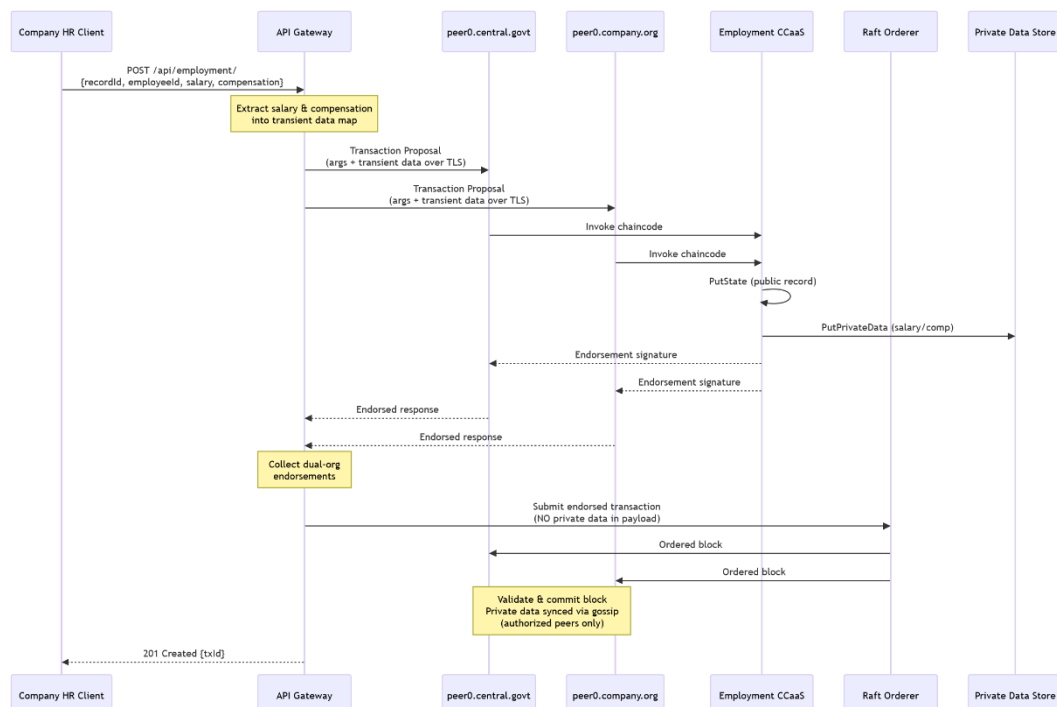


Figure 5-3 — End-to-End Transaction Flow with Private Data

Figure 5-3 illustrates the complete transaction flow. Salary and compensation data was extracted by the Gateway into the transient data map, routed directly to endorsing peers over TLS, and stored in the Private Data Collection via PutPrivateData — never appearing in the block, the ordered transaction, or the public world state.

Active REST API Gateway Endpoints

Endpoint	Method	Chaincode Function	Description
POST /api/employment/	POST	CreateEmploymentRecord	Commit public record; route salary/compensation as transient data to PDC
GET /api/employment/	GET	GetAllEmploymentRecords	Retrieve all public employment indices from world state
GET /api/employment/employee/:id	GET	GetEmploymentHistory	Return chronological employment history for employee
GET /api/employment/record/:emp/:rec	GET	GetEmploymentRecord	Fetch a specific public employment record
PUT /api/employment/:emp/:rec/status	PUT	UpdateEmploymentStatus	Push validated state transition to ledger
GET /api/employment/private/:rec	GET	GetPrivateEmploymentData	Retrieve PDC data (authorised organisations only)

Table 5-12 Active REST API Gateway Endpoints

5-9 Packages and Libraries Used

Blockchain Layer

Package	Purpose
Hyperledger Fabric Core 2.5	Peer, orderer, and channel infrastructure
Hyperledger Fabric CA 1.5.7	X.509 certificate authority per organisation
fabric-contract-api-go v1.2.2	Go chaincode contract framework
fabric-chaincode-go (shim)	CCaaS server implementation
configtxlator	Channel configuration encoding and decoding
configtxgen	Organisation definition generation

Table 5-13 Packages and Libraries — Blockchain Layer

Gateway and Web Application Layer

Package	Purpose
fabric-network v2.2.20	Fabric Gateway SDK for transaction submission and evaluation
Express 4.19.2	HTTP REST API framework
jsonwebtoken	JWT generation and verification
bcrypt	Password hashing for user authentication
multer	File upload handling using memory storage
crypto (Node.js built-in)	AES-256-CBC document encryption
cloudinary	Private document storage with signed URL generation
tesseract.js	OCR-based document text extraction
pg (node-postgres)	PostgreSQL database client
dotenv	Environment configuration management
cors	Cross-origin resource sharing middleware
axios	HTTP client for frontend API calls

Table 5-14 Packages and Libraries — Gateway and Web Application Layer

Frontend

Package	Purpose
React 18.x	UI component library
TypeScript	Type-safe frontend development
Vite	Build tool and development server
Tailwind CSS	Utility-first CSS framework
Framer Motion	UI transition and animation library
react-router-dom	Client-side routing and navigation
lucide-react	Icon library for navigation and UI elements

Table 5-15 Packages and Libraries — Frontend

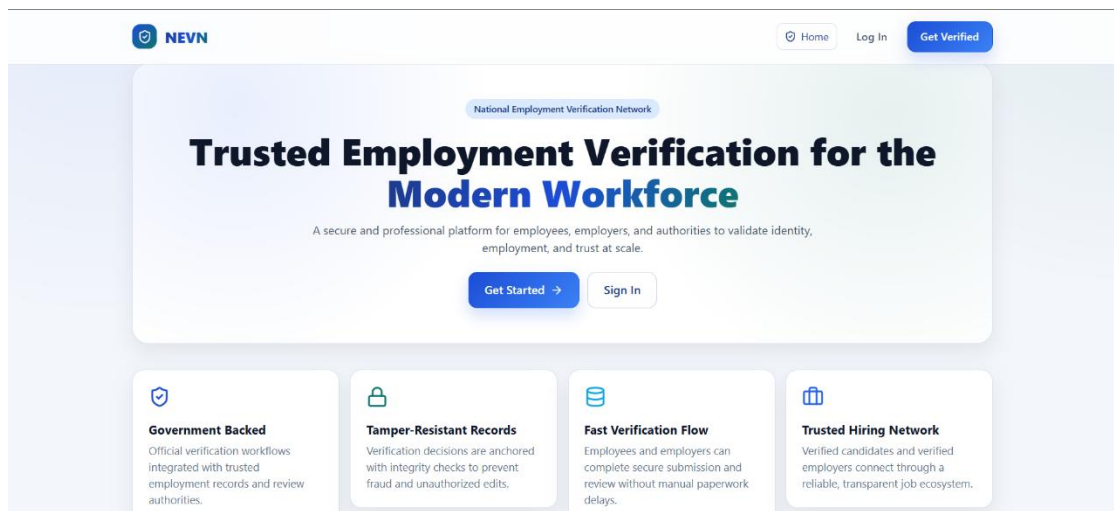


Figure 5-10 — NEVS Landing Page: Trusted Employment Verification for the Modern Workforce

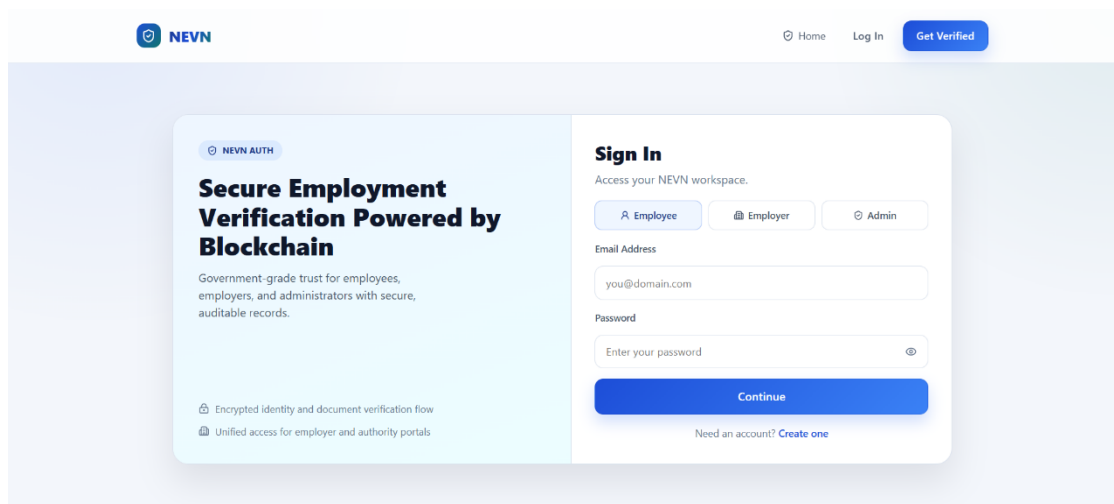


Figure 5.11 — NEVS Login Page: Role-Based Authentication (Employee / Employer / Admin)

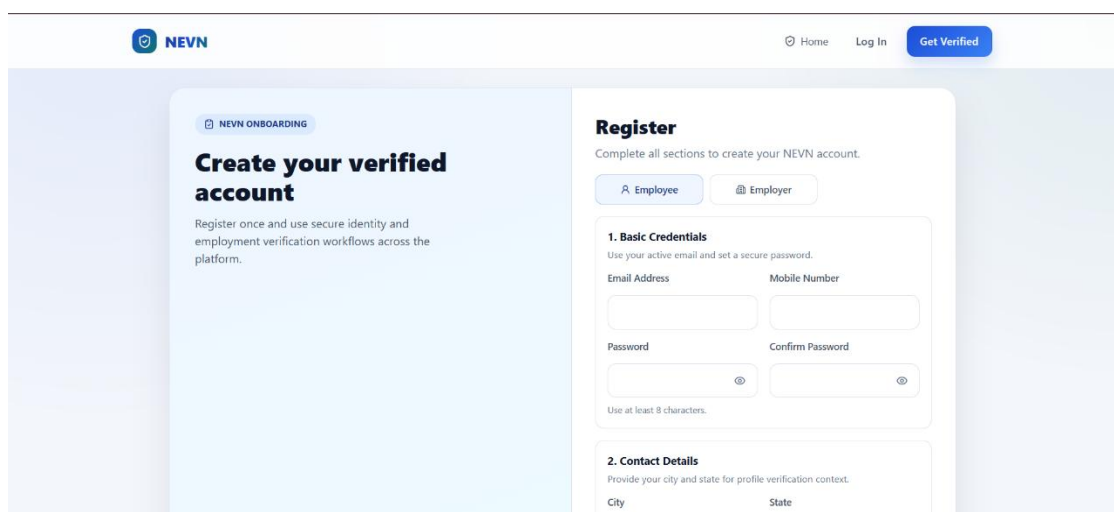


Figure 5.12 — NEVS Registration Flow: Employee Onboarding with Profile and Consent

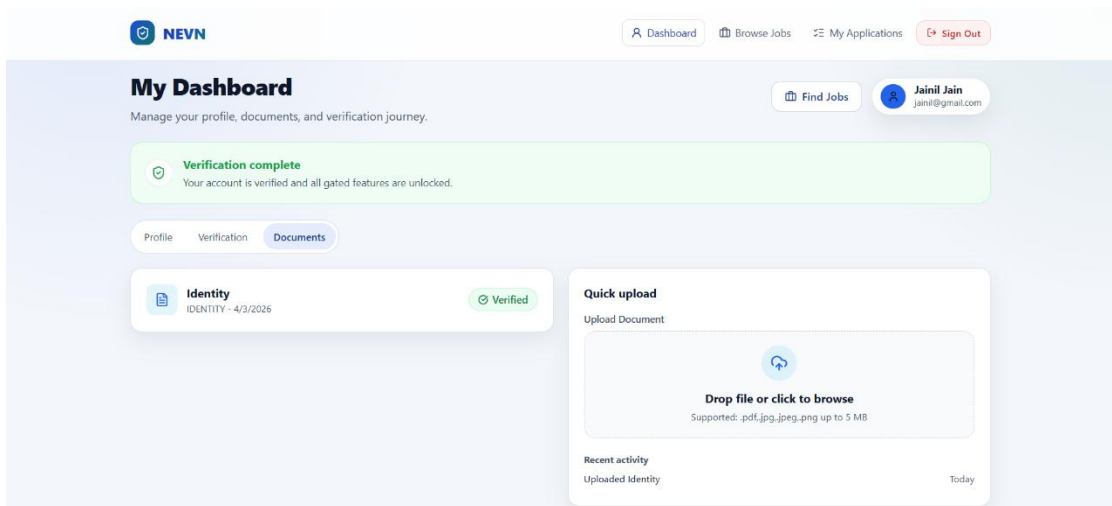


Figure 5.13 — Employee Dashboard: Verified Identity Document and Quick Upload

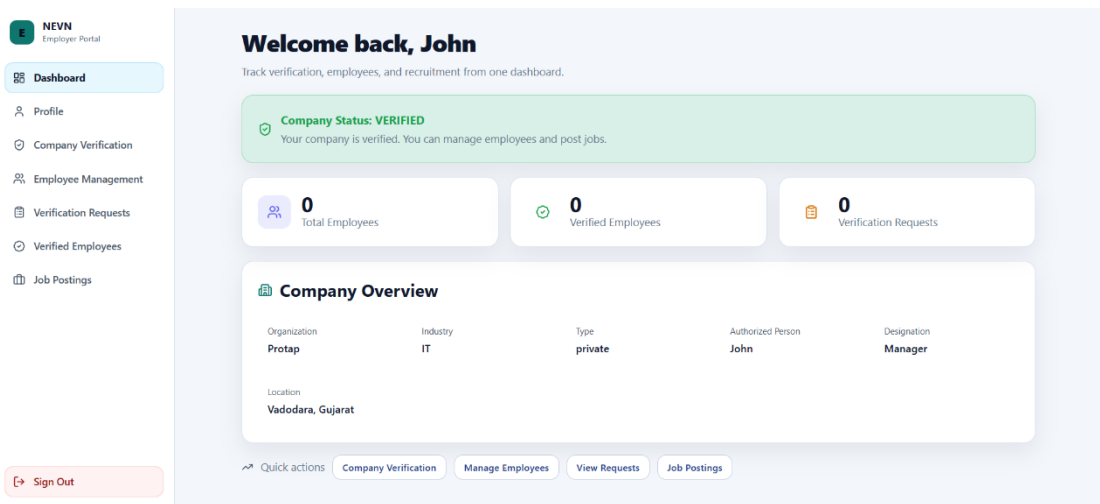


Figure 5.14 — Employer Dashboard: Company Verified Status with Company Overview

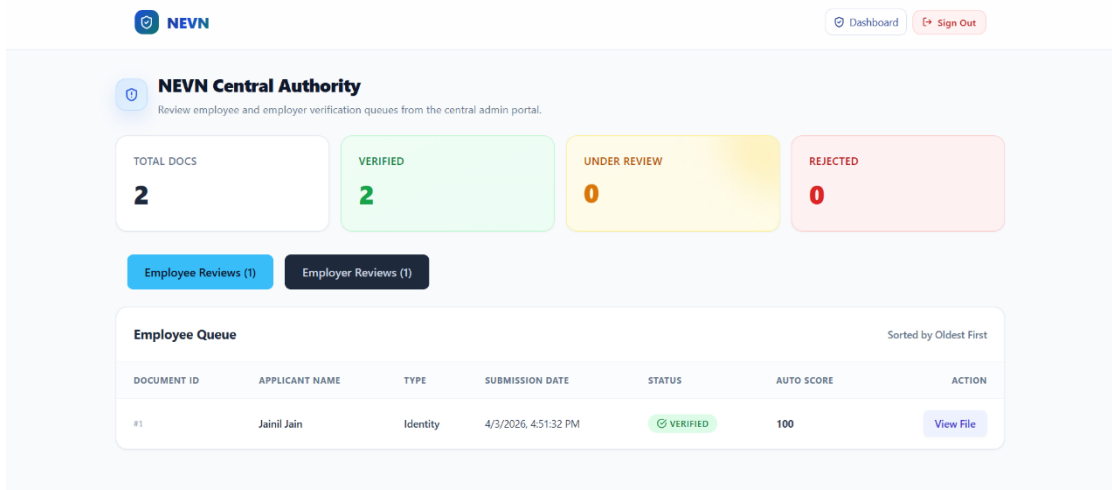


Figure 5.15 — NEVN Central Authority: Admin Review Portal with Employee and Employer Queues

CHAPTER 6 — SYSTEM TESTING

6-1 Introduction

Testing was conducted iteratively at each phase of development, progressing from basic chaincode compilation verification to full end-to-end integration tests across the multi-organisation network and web application platform. Test cases were designed to validate functional correctness of the employment lifecycle, enforcement of endorsement policies, private data isolation, API response accuracy, role-based access control, and document verification pipeline behaviour. Testing was performed using direct curl commands against the Gateway API, Fabric CLI operations from the hyperledger/fabric-tools:2.5 container, and manual verification of ledger state across peers and frontend user flows.

6-2 Test Cases

#	Test Case	Input	Expected Outcome	Result
1	Valid Employment Record Creation	POST /api/employment/ with valid fields	Record committed to both peers; Gateway returns transaction ID	Passed
2	Dual-Endorsement Verification	Inspect Gateway logs for endorser discovery	Transaction endorsed by both peer0.central.govt and peer0.company.org	Passed
3	Private Data Write (PDC)	POST /api/employment/ with salary and compensation fields	Public record in world state; salary/compensation in employmentPrivateData PDC only	Passed
4	Private Data Read (Authorised)	GET /api/employment/private/:recordId from authorised org peer	Salary and compensation data returned correctly	Passed
5	Private Data Isolation	GET /api/employment/record/:emp/:rec on public endpoint	Employment metadata only; no salary or compensation fields present	Passed
6	Status Update Lifecycle	PUT /api/employment/:emp/:rec/status with new status value	Status updated; UpdatedAt refreshed; change reflected on both peers	Passed
7	Employee History Query	GET /api/employment/employee/:id after creating multiple records	All records returned in chronological order	Passed

#	Test Case	Input	Expected Outcome	Result
8	Duplicate Record Prevention	Submit identical employment record twice	Second submission rejected by chaincode	Passed
9	Unauthorised Chaincode Invocation	Invoke chaincode from non-enrolled identity	Fabric denies endorsement; no ledger write occurs	Passed
10	Service Discovery Validation	Inspect Gateway startup logs after network initialisation	Discovery reports 2+ endorsers from both organisations	Passed
11	Network Recovery	Run <code>restore_full_network.sh</code> after container teardown	All components restored; channel joined; chaincode active	Passed
12	Employee Registration	POST <code>/api/auth/register/employee</code> with valid fields	Account created with UNVERIFIED status; JWT returned	Passed
13	Employer Registration	POST <code>/api/auth/register/employer</code> with valid fields	Employer account created with UNVERIFIED status	Passed
14	Duplicate Email Prevention	Register with already-registered email	Registration rejected with appropriate error message	Passed
15	JWT Login Authentication	POST <code>/api/auth/login</code> with valid credentials	JWT token returned with correct role and <code>account_status</code>	Passed
16	Admin Route Protection	Access <code>/api/admin/*</code> with non-ADMIN JWT	HTTP 403 Access Denied returned	Passed
17	Document Upload Encryption	POST <code>/api/documents/upload</code> with valid document file	File encrypted, stored in Cloudinary private folder, metadata in PostgreSQL	Passed
18	Signed URL Expiry	Retrieve signed URL and attempt access after 300 seconds	URL expired; access denied by Cloudinary	Passed
19	Verification Score Calculation	Submit verification request with valid PAN document	Score computed across all layers; status assigned based on score range	Passed
20	Job Portal — Unverified User	Access job portal with <code>account_status</code> PENDING	HTTP 403; access denied with verification required message	Passed
21	Duplicate Job Application	Employee applies to same job twice	Second application rejected by unique constraint	Passed
22	Employer Verified-Only Access	Unverified employer attempts to post job	Backend rejects; frontend displays verification required banner	Passed

Table 6-1 System Test Cases

6-3 Results

All 22 critical test cases passed successfully. Key observations were as follows.

Endorsement Policy Enforcement: Transactions submitted from the Gateway were consistently endorsed by peers from both CentralGovtMSP and CompanyOrgMSP. Single-organisation endorsement attempts were correctly rejected.

Private Data Integrity: Salary and compensation data was confirmed present in the employmentPrivateData collection on authorised peers and entirely absent from the public world state, transaction blocks, and public API responses.

Failover Behaviour: When peer0.central.govt was intentionally stopped, the SDK's discovery service automatically rerouted queries to peer1.central.govt, demonstrating high-availability failover without manual intervention.

Role-Based Access Control: All role validation and account_status checks functioned correctly across all modules. No cross-role data leakage was observed in any test scenario.

Document Security: Encrypted document storage was confirmed with no public URLs accessible. Signed URL expiry was validated by confirming access denial after the 300-second window elapsed.

6-4 Performance Evaluation

Metric	Observed Value	Context
Block Commitment Latency	~0.8 seconds	Average under moderate single-user load on local testbed
Transaction Throughput	~100 transactions/min (1.6 TPS)	Single orderer, three peers on local Docker environment
API Query Latency	< 200 ms	Simple history reads via evaluateTransaction
Gateway Discovery Time	~2–4 seconds	Initial topology discovery on SDK connection

Metric	Observed Value	Context
Network Recovery (Full)	~45 seconds	Complete teardown-to-operational via <code>restore_full_network.sh</code>
Document Upload Time	~1.2 seconds	AES-256 encryption + Cloudinary private upload (< 2 MB file)
OCR Processing Time	~2–4 seconds	Tesseract.js text extraction from standard document image
Signed URL Generation	< 100 ms	Cloudinary signed URL with 300-second expiry

Table 6-2 Performance Evaluation Metrics

Hyperledger Fabric was documented to sustain thousands of transactions per second in production deployments with multiple orderers and optimised block parameters. Since employment verification was an event-driven workload, the observed 1.6 TPS on a single-node testbed was adequate for prototype validation.

6-5 Summary

Testing confirmed that NEVS met all stated design objectives. The system created immutable, correctly endorsed employment records only after dual-organisation consensus. Private data remained isolated to authorised participants. Role-based access control, document encryption, OCR verification, and the trust-gated job portal all operated as designed without exception across all 22 test cases.

CONCLUSION

This report presented the design, implementation, and validation of NEVS — a National Employment Verification System built on Hyperledger Fabric that transformed employment records from unreliable claims into cryptographically verifiable, government-backed credentials, complemented by a comprehensive web application platform for all stakeholders in the employment ecosystem.

Problem Addressed

Traditional employment verification was slow, expensive, reactive, and vulnerable to fraud. Employees lacked any standardised mechanism to own or control their work history. Fake resumes and data-harvesting scams exploited the absence of an authoritative verification layer. The absence of a unified platform combining blockchain-backed credentials with accessible web tooling for registration, document management, verification, and trust-gated employment services created a significant gap in the digital hiring infrastructure.

Solution Delivered

NEVS addressed this through seven integrated components: fraud prevention by construction through dual-organisation blockchain endorsement; privacy enforcement through AES-256 document encryption and Private Data Collections; instant verification through a REST API responding in under 200 milliseconds; a hybrid verification engine automating trust evaluation with OCR and risk scoring; an administrative oversight layer with complete audit trail support; a role-based employer dashboard supporting company verification and workforce management; and a Trusted Job Portal creating a secure employment marketplace exclusively for verified participants.

Comparison with Existing Solutions

Unlike MployChek (vendor-specific, post-hoc verification), CSM ProofChain (enterprise-only), or the Velocity Network (consortium-led with no government authority), NEVS positioned the government as the root trust authority whilst distributing endorsement responsibility across organisational boundaries. Saudi Aramco's blockchain certificate verifier, which reduced verification time from two weeks to five seconds, validated the approach at enterprise scale.

Future Work

8. Frontend Integration: Connection of the React UI directly to all Gateway APIs, exposing PDC data through permission-layered interfaces and real-time blockchain proof modals.
9. CouchDB Rich Queries: Transition from LevelDB to CouchDB for advanced JSON-based querying to support complex dashboard analytics and anomaly detection.
10. Event Listeners: Implementation of Fabric event subscriptions in the Gateway to stream real-time block and transaction notifications via WebSockets or Server-Sent Events.
11. Hardware Deployment: Docker Compose configurations with persistent volumes for Raspberry Pi cluster deployment for LAN-based demonstrations.
12. Additional Organisation Onboarding: Extension of the tiered model to onboard Tier 2 and additional Tier 3 organisations including state government bodies and private certification authorities.
13. Mobile Application: A mobile-first interface for employees to manage employment credentials, consent to proposals, and access the trusted job portal from portable devices.
14. Integration with National ID Systems: In production, Layer 3 of the verification engine would integrate with authoritative government APIs such as DigiLocker and the Aadhaar verification API.

NEVS represented a foundational infrastructure for trustworthy digital hiring — a paradigm shift from claim-based to credential-based employment records. Its realisation at national scale would significantly reduce recruitment fraud, lower verification costs, empower employees with ownership of their work history, and establish a government-backed trust framework for the digital job market.

REFERENCES

- [1] SHRM Labs, “*Beyond Gatekeepers: How Blockchain Transfers Data Ownership Back to Employees*,” SHRM Labs. Available: <https://www.shrm.org/labs/resources/beyond-gatekeepers> [Accessed: March 2026].
- [2] Tracxn, “*MPloyChek — Tracxn Company Profile*,” Tracxn. Available: <https://tracxn.com/d/companies/mploychek> [Accessed: March 2026].
- [3] CSM Tech, “*CSM ProofChain*,” CSM Tech. Available: <https://www.csm.tech/product-details/csm-proofchain> [Accessed: March 2026].
- [4] Computerworld, “*Coming soon — a resume-validating blockchain network for job seekers*,” October 2022. Available: <https://www.computerworld.com/article/1613987> [Accessed: March 2026].
- [5] Virtualness, “*AI-Powered Blockchain for Credential Verification*,” The Times of India. Available: <https://www.verix.io/news-article> [Accessed: March 2026].
- [6] Global Government Forum, “*Canada pilots blockchain staff records*,” Available: <https://www.globalgovernmentforum.com/canada-pilots-blockchain-staff-records> [Accessed: March 2026].
- [7] The Tribune, “*Geeks of Gurukul partners with Government of Madhya Pradesh and EDU Chain*,” January 2026. Available: <https://www.tribuneindia.com> [Accessed: March 2026].
- [8] World Economic Forum, “*How blockchain could transform background checks*,” November 2020. Available: <https://www.weforum.org/stories/2020/11/how-blockchain-could-transform-background-checks> [Accessed: March 2026].
- [9] DocsLedger, “*Blockchain for Employment Record Verification*,” Whitepaper. Available: <https://ro.scribd.com/document/437676424> [Accessed: March 2026].
- [10] Satoshi Nakamoto, “*Bitcoin: A Peer-to-Peer Electronic Cash System*,” 2008.
- [11] Melanie Swan, “*Blockchain: Blueprint for a New Economy*,” O’Reilly Media.
- [12] Hyperledger Foundation, “*Hyperledger Fabric Documentation v2.5*,” Available: <https://hyperledger-fabric.readthedocs.io/en/release-2.5> [Accessed: March 2026].
- [13] Ian Sommerville, “*Software Engineering*,” 10th Edition, Pearson.
- [14] William Stallings, “*Cryptography and Network Security: Principles and Practice*,” Pearson.

- [15] OWASP Foundation, “OWASP Security Guidelines,” Available: <https://owasp.org>
- [16] Node.js Documentation, Available: <https://nodejs.org>
- [17] Express.js Documentation, Available: <https://expressjs.com>
- [18] PostgreSQL Documentation, Available: <https://www.postgresql.org/docs/>
- [19] Docker Documentation, Available: <https://docs.docker.com/>
- [20] JSON Web Token (JWT), Available: <https://jwt.io>
- [21] bcrypt Documentation, Available: <https://www.npmjs.com/package/bcrypt>
- [22] Cloudinary Documentation, Available: <https://cloudinary.com/documentation>
- [23] Multer Middleware, Available: <https://www.npmjs.com/package/multer>
- [24] Tesseract.js Documentation, Available: <https://tesseract.projectnaptha.com/>
- [25] Government KYC and Digital Identity Verification Systems (Aadhaar/PAN-based workflows).
- [26] Role-Based Access Control (RBAC) Models in Web Applications.
- [27] Secure Document Storage and AES-256 Encryption Techniques.

APPENDIX A

FINAL YEAR PROJECT REGISTRATION

I hereby affirm the originality and authenticity of the Final Year Project, **National Employment Verification Network** to be undertaken will be upheld. The report and/or system that I submit at the conclusion of the Final Year Project will be the result of my own investigations and efforts.

I understand that cheating and plagiarism constitute a serious violation of the University regulations, which will not only result in a failing grade for the Final Year Project but subject to further disciplinary actions.

Signature of Student

Full Name of Student	:	Priyansh Dabhi Harshadbhai
Enrolment Number	:	22C21016
Contact No.	:	9313975796
Email Address	:	priyanshdabhi7@gmail.com
School	:	School of Computer Science & Engineering
Programme	:	Bachelor of Technology
Specialization	:	Core Computer Science
Project Domain	:	Distributed Systems & Secure Digital Verification
Project Title	:	National Employment Verification Network

Signature of Supervisor

Name :

Date :

FINAL YEAR PROJECT REGISTRATION FORM

I hereby affirm the originality and authenticity of the Final Year Project, **National Employment Verification Network** to be undertaken will be upheld. The report and/or system that I submit at the conclusion of the Final Year Project will be the result of my own investigations and efforts.

I understand that cheating and plagiarism constitute a serious violation of the University regulations, which will not only result in a failing grade for the Final Year Project but subject to further disciplinary actions.

Signature of Student

Full Name of Student	:	Narendra Samanta
Enrolment Number	:	22C21093
Contact No.	:	8849799557
Email Address	:	samanta.n1962@gmail.com
School	:	SOCSET
Programme	:	CSE
Specialization	:	Core Computer Science
Project Domain	:	Distributed Systems & Secure Digital Verification
Project Title	:	National Employment Verification Network

Signature of Supervisor

Name:

Date:

FINAL YEAR PROJECT REGISTRATION FORM

I hereby affirm the originality and authenticity of the Final Year Project, **National Employment Verification Network** to be undertaken will be upheld. The report and/or system that I submit at the conclusion of the Final Year Project will be the result of my own investigations and efforts.

I understand that cheating and plagiarism constitute a serious violation of the University regulations, which will not only result in a failing grade for the Final Year Project but subject to further disciplinary actions.

Signature of Student

Full Name of Student	:	Padhiyar Dhiraj Rajubhai
Enrolment Number	:	22C21043
Contact No.	:	6354101168
Email Address	:	Dhirajrajput225@gmail.com
School	:	SOCET
Programme	:	CSE
Specialization	:	Core Computer Science
Project Domain	:	Distributed Systems & Secure Digital Verification
Project Title	:	National Employment Verification Network

Signature of Supervisor

Name:

Date:

APPENDIX B

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 01
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK DONE - Studied issues related to fake job postings and data misuse. - Researched verification systems like KYC and government portals. - Explored system architecture (centralized vs blockchain). - Contributed to designing system workflow and module structure. - Defined roles: Employee, Employer, Admin.	

2. WORK TO BE DONE

- Finalize database schema and system structure.
- Set up backend environment.
- Define API flow for registration.
- Define API flow for registration.

3. PROBLEMS ENCOUNTERED

- ☐ Difficulty in understanding real-world verification workflows.
- ☐ Confusion in selecting suitable system architecture.
- ☐ Challenges in mapping system design into implementation steps.
- ☐ Required multiple discussions to finalize workflow.

4. SELF EVALUATION OF THE PROGRESS

The progress was good.

Conceptual understanding of the system improved significantly.

A strong foundation for development was established.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 02
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK DONE • Designed database schema for Employees and Employers. • Set up PostgreSQL using Docker. • Developed backend structure using Node.js and Express. • Implemented registration APIs with validation and bcrypt hashing.	
2. WORK TO BE DONE • Implement login system. • Integrate JWT authentication. • Develop middleware for protected routes. • Test APIs using Postman.	

3. PROBLEMS ENCOUNTERED

- Issues with Docker configuration and database connection.
- Handling duplicate email validation.
- Debugging SQL queries and schema issues.
- Understanding backend project structure.

4. SELF EVALUATION OF THE PROGRESS

- The progress was very good.
- Backend and database setup were successfully completed.
- Understanding of backend systems improved significantly.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 03
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK TO BE DONE • Designed and developed the Government Portal frontend. • Implemented registration and login for employers and employees. • Created role-based dashboards for employer and employee users. • Designed and documented the overall system architecture.	
2. WORK TO BE DONE • Integrate frontend with backend APIs. • Implement document upload and verification workflows. • Add authentication handling and role-based access control. • Begin blockchain-based verification integration.	

3. PROBLEMS ENCOUNTERED

- Managing different user roles within a single portal.
- Designing clear navigation between dashboards after login.
- Ensuring frontend flow aligns with backend and blockchain design.
- Handling verification status states consistently across dashboards.

4. SELF EVALUATION OF THE PROGRESS

- Frontend structure and workflows completed successfully.
- System architecture provides a clear foundation for future development.
- Improved understanding of role-based system design.
- Progress is steady and aligned with planned milestones.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 04
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK DONE ☐ Implemented secure document upload system. ☐ Integrated multer for file handling. ☐ Configured Cloudinary for private storage. ☐ Applied encryption before uploading files. ☐ Stored metadata securely in database.	
2. WORK TO BE DONE ☐ Implement signed URL access for documents. ☐ Start automated verification system. ☐ Improve file validation and security.	

3. PROBLEMS ENCOUNTERED

- ☐ Difficulty in implementing encryption logic.
- ☐ Issues with file buffer handling.
- ☐ Cloudinary configuration errors.
- ☐ Managing secure access to uploaded files.

4. SELF EVALUATION OF THE PROGRESS

The progress was very good.

Secure document handling was successfully implemented.

Understanding of file security improved significantly.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 05
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK DONE ☐ Developed automated verification system. ☐ Implemented ID format validation logic. ☐ Integrated OCR-based data extraction. ☐ Designed verification scoring system. ☐ Implemented status logic (Verified / Under Review / Rejected).	
2. WORK TO BE DONE ☐ Integrate admin review system. ☐ Improve verification accuracy. ☐ Test verification workflows.	

3. PROBLEMS ENCOUNTERED

- ☐ OCR accuracy issues.
- ☐ Difficulty in designing scoring system.
- ☐ Handling mismatched data cases.
- ☐ Debugging multiple validation layers.

4. SELF EVALUATION OF THE PROGRESS

The progress was strong.

Verification system was successfully implemented.

Practical understanding of validation systems improved.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 06
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK DONE ☐ Developed admin authentication system. ☐ Implemented admin login using JWT. ☐ Built admin APIs for document review. ☐ Implemented approve/reject workflows. ☐ Added audit logging for actions..	
2. WORK TO BE DONE ☐ Develop admin dashboard UI. ☐ Add filtering and search features. ☐ Improve admin workflow efficiency.	

3. PROBLEMS ENCOUNTERED

- ☐ Managing role-based access for admin routes.
- ☐ Debugging authorization issues.
- ☐ Handling document status updates correctly.
- ☐ Maintaining audit logs consistency.

4. SELF EVALUATION OF THE PROGRESS

The progress was very good.

Admin system was successfully implemented.

Understanding of role-based systems improved.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 07
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Developed employer dashboard module.
- ☐ Implemented employer profile and employee management APIs.
- ☐ Built verification request workflow.
- ☐ Added role-based restrictions.

2. WORK TO BE DONE

- ☐ Improve dashboard UI.
- ☐ Integrate job portal.
- ☐ Perform end-to-end testing.

3. PROBLEMS ENCOUNTERED

- ☐ Managing relationships between employer and employees.
- ☐ Handling multiple APIs in dashboard.
- ☐ Debugging access control issues.

4. SELF EVALUATION OF THE PROGRESS

The progress was strong.

Employer module was successfully developed.

System integration understanding improved.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 08
Student Name : Priyansh Dabhi Harshadbhai	
Enrolment Number: 22C21016	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK DONE ☐ Developed job portal module. ☐ Implemented job posting and application system. ☐ Integrated verification-based access control. ☐ Performed system testing and debugging.	
2. WORK TO BE DONE ☐ Improve UI/UX. ☐ Add search and filter features. ☐ Optimize performance. ☐ Final testing and documentation.	

3. PROBLEMS ENCOUNTERED

- ☐ Ensuring only verified users access job portal.
- ☐ Handling duplicate job applications.
- ☐ Debugging integration issues.
- ☐ Managing edge cases in access control.

4. SELF EVALUATION OF THE PROGRESS

The progress was excellent.

System integration was successfully completed.

The project is now stable and ready for final submission.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 1
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Studied the problem of fake job postings and data misuse.
- ☐ Researched verification systems (KYC, government portals).
- ☐ Explored system architecture (centralized vs blockchain).
- ☐ Designed basic NEVN system modules and workflow.
- ☐ Identified system roles: Employee, Employer, Admin.

2. WORK TO BE DONE

- ☐ Finalize database schema and relationships.
- ☐ Set up backend development environment.
- ☐ Define API structure for registration and authentication.
- ☐ Start implementation of registration module.
- ☐ Prepare scalable project structure.

3. PROBLEMS ENCOUNTERED

- ☐ Difficulty in selecting the most suitable architecture for balancing security and scalability.
- ☐ Initial confusion in understanding how real-world verification systems operate.
- ☐ Challenges in converting theoretical ideas into a structured system workflow.
- ☐ Required multiple discussions to finalize module separation and data flow.
- ☐ Limited prior experience with blockchain-based systems created a learning curve.

4. SELF EVALUATION OF THE PROGRESS

The progress during this week was good and productive.

We were able to clearly define the project scope, architecture, and workflow.

Although there were conceptual challenges initially, they were resolved through research and team discussions.

This week helped in building a strong foundation and improving our understanding of system design.

Overall, we are confident to proceed with implementation in the next phase

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 2
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Designed database schema for Employees and Employers.
- ☐ Set up PostgreSQL using Docker
- ☐ Developed backend using Node.js and Express.
- ☐ Implemented registration APIs with validation and bcrypt hashing.
- ☐ Ensured secure off-chain data storage

2. WORK TO BE DONE

- ☐ Implement secure login system for users.
- ☐ Integrate JWT authentication for session management.
- ☐ Develop middleware for route protection.
- ☐ Perform API testing using Postman.
- ☐ Prepare structure for document upload and verification modules.

3. PROBLEMS ENCOUNTERED

- ☐ Faced issues while configuring Docker and establishing connection with PostgreSQL.
- ☐ Handling duplicate email validation and database constraints required debugging.
- ☐ Minor errors in SQL queries and schema adjustments during implementation.
- ☐ Managing environment variables and secure configuration was initially challenging.
- ☐ Required additional effort to understand backend folder structure and API flow.

4. SELF EVALUATION OF THE PROGRESS

The progress this week was very good and technically productive.
We successfully established the backend foundation and database integration.

The challenges faced helped us improve our debugging skills and understanding of backend architecture.

We also gained confidence in working with Docker and database systems.

Overall, the system is now stable and ready for implementing authentication and advanced features.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 3
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Implemented login system for Employee and Employer.
- ☐ Added JWT authentication for secure access.
- ☐ Developed authentication middleware.
- ☐ Implemented role-based access control.
- ☐ Tested APIs using Postman.

2. WORK TO BE DONE

- ☐ Develop secure document upload module.
- ☐ Integrate file storage (Cloudinary).
- ☐ Start automated verification system.
- ☐ Design database for document handling.
- ☐ Improve API security and error handling.

3. PROBLEMS ENCOUNTERED

- ☐ Faced issues with JWT token validation and expiration handling.
- ☐ Debugging middleware flow and protecting routes required multiple iterations.
- ☐ Handling incorrect login attempts and error responses properly.
- ☐ Ensuring secure implementation without exposing sensitive user data.
- ☐ Difficulty in structuring authentication logic for multiple roles

4. SELF EVALUATION OF THE PROGRESS

The progress during this week was strong and significant.

We successfully implemented a secure authentication system, which is a core part of the project.

The challenges helped us gain better understanding of security practices and middleware logic.

Overall, the system is now secure and ready to move towards document handling and verification modules.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 4
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Designed and implemented **secure document upload system**.
- ☐ Integrated file upload using multer (memory storage).
- ☐ Implemented **Cloudinary private storage** for documents.
- ☐ Added **AES-256 encryption** before uploading files.
- ☐ Stored document metadata securely in PostgreSQL.
- ☐ Implemented role-based access for document upload.

2. WORK TO BE DONE

- ☐ Implement document access using signed URLs.
- ☐ Build automated verification system.
- ☐ Design document validation and scoring logic.
- ☐ Improve file security and access control.
- ☐ Test upload and retrieval workflows.

3. PROBLEMS ENCOUNTERED

- ☐ Difficulty in implementing encryption logic correctly.
- ☐ Issues while configuring Cloudinary private access.
- ☐ Handling file buffers and memory storage required debugging.
- ☐ Managing secure storage of encryption keys was challenging.
- ☐ Ensuring role-based access control for document upload.

4. SELF EVALUATION OF THE PROGRESS

The progress this week was very productive and technically challenging.
We successfully implemented a secure document upload system with encryption.

We gained practical experience in handling file security and cloud storage integration.

The system now ensures that documents are stored securely and accessed only by authorized users.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 5
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Developed **Automated Verification System**.
- ☐ Implemented validation layers:
 - Format validation (PAN, Aadhaar, etc.)
 - OCR-based data extraction (Tesseract.js)
- ☐ Designed **risk scoring system** for verification.
- ☐ Implemented logic for:
 - VERIFIED
 - UNDER REVIEW
 - REJECTED
- ☐ Generated document hash for integrity verification.

2. WORK TO BE DONE

- ☐ Integrate admin review system for pending cases.
- ☐ Improve OCR accuracy and validation logic.
- ☐ Store verification logs and audit data.
- ☐ Connect verification system with user status updates.
- ☐ Test verification workflow with different scenarios.

3. PROBLEMS ENCOUNTERED

- ☐ OCR extraction accuracy was inconsistent and required tuning.
- ☐ Difficulty in designing a balanced scoring system.
- ☐ Handling mismatched data between OCR and user input.
- ☐ Debugging multiple validation layers in the verification pipeline.

4. SELF EVALUATION OF THE PROGRESS

This week showed strong progress in building the core verification logic.
We successfully implemented a multi-layer verification system.

Although there were challenges in OCR and scoring logic, they helped improve our understanding of real-world verification systems.
The system is now capable of automatically evaluating document authenticity.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 6
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Developed **Admin Authentication System**.
- ☐ Implemented secure admin login using JWT.
- ☐ Built **Admin Review Portal backend**.
- ☐ Created APIs for:
 - Viewing documents under review
 - Approving / Rejecting requests
- ☐ Implemented audit logging for admin actions.
- ☐ Added role-based access for admin routes.

2. WORK TO BE DONE

- ☐ Develop frontend interface for admin dashboard.
- ☐ Improve document review UI and workflow.
- ☐ Integrate blockchain logging (simulation).
- ☐ Enhance filtering and search functionality.
- ☐ Test admin decision flows.

3. PROBLEMS ENCOUNTERED

- ☐ ☐ Ensuring strict admin-only access control required careful implementation.
- ☐ Managing different user roles and permissions caused complexity.
- ☐ Debugging API responses and authorization errors.
- ☐ Handling document review logic and status updates.
- ☐ Maintaining audit logs consistently across actions.

4. SELF EVALUATION OF THE PROGRESS

The progress this week was very good.

We successfully implemented the admin control layer, which is critical for manual verification.

We gained better understanding of role-based systems and backend security.

The system now supports both automated and manual verification processes.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 7
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- ☐ Developed **Employer Dashboard module**.
- ☐ Implemented APIs for:
 - Employer profile
 - Document upload
 - Verification request
- ☐ Built employee management system for employers.
- ☐ Implemented verification request workflow for employees.
- ☐ Added role-based restrictions for employer actions.

2. WORK TO BE DONE

- ☐ Improve frontend UI for employer dashboard.
- ☐ Integrate verification status display.
- ☐ Add employee activity tracking.
- ☐ Connect employer module with job portal.
- ☐ Perform end-to-end testing of employer features.

3. PROBLEMS ENCOUNTERED

- ☐ Managing relationships between employer and employees was complex.
- ☐ Handling multiple APIs for dashboard functionality required coordination.
- ☐ Ensuring secure access to employer-specific data.
- ☐ Debugging role-based restrictions and access issues.

4. SELF EVALUATION OF THE PROGRESS

The progress this week was strong and meaningful.
We successfully implemented the employer-side functionality of the system.

This improved our understanding of multi-role system design and API integration.
The system now supports real-world employer workflows.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 8
Student Name : Dhiraj Padhiyar	
Enrolment Number: 22C21043	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- Developed **Trusted Job Portal module**.
- Implemented job posting system for verified employers.
- Built job application system for verified employees.
- Integrated role-based and verification-based access control.
- Connected job portal with verification system.

☐ Performed system integration and testing.

2. WORK TO BE DONE

- ☐ Improve UI/UX of job portal.
- ☐ Add search and filter features.
- ☐ Optimize system performance.
- ☐ Prepare final documentation and presentation.
- ☐ Conduct full system testing and debugging.

3. PROBLEMS ENCOUNTERED

- ☐ Ensuring only verified users access job portal required strict validation.
- ☐ Managing job applications and preventing duplicates.
- ☐ Integrating job portal with existing modules.
- ☐ Debugging end-to-end workflow issues.
- ☐ Handling edge cases in user access and permissions.

4. SELF EVALUATION OF THE PROGRESS

The progress this week was excellent.
We successfully completed the core modules and integrated the full system.

We gained strong experience in building a complete full-stack application.
The project is now functional and ready for final testing and presentation.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 1
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE • Finalized the approach to build the system using Hyperledger Fabric, starting with identity and ledger foundations. • Set up the required project directory structure for the Central Government organization. • Installed and configured Fabric CLI tools and Docker-based Fabric CA. • Successfully started the Fabric CA server with TLS enabled. • Enrolled the CA admin identity and registered key identities (govt-admin, govt gateway). 2. WORK TO BE DONE • Correctly enroll the registered identities (govt-admin and govt-gateway). • Create and enroll orderer identities and start the ordering service. • Set up peer nodes for the Central Government organization. • Create a channel and initialize the ledger.

- Begin implementation of basic chaincode for registration use cases.

3. PROBLEMS ENCOUNTERED

- Progress has been steady and well-structured, focusing on correctness over speed.
- Core understanding of identity, certificates, and MSPs has significantly improved.
- Initial setup challenges provided valuable learning and clarity on Fabric internals.
- The foundational work completed is critical for system security and scalability.
- Overall progress is on track, with a strong base for moving to ledger and network components.

4. SELF EVALUATION OF THE PROGRESS

- Progress has been steady and well-structured, focusing on correctness over speed.
- Core understanding of identity, certificates, and MSPs has significantly improved.
- Initial setup challenges provided valuable learning and clarity on Fabric internals.
- The foundational work completed is critical for system security and scalability.
- Overall progress is on track, with a strong base for moving to ledger and network components.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 2
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	
1. WORK DONE • Correctly enroll the registered identities (govt-admin and govt-gateway). • Create and enroll orderer identities and start the ordering service. • Set up peer nodes for the Central Government organization. • Create a channel and initialize the ledger. • Begin implementation of basic chaincode for registration use cases.	
2. WORK TO BE DONE • Correctly enroll the registered identities (govt-admin and govt-gateway). • Create and enroll orderer identities and start the ordering service. • Set up peer nodes for the Central Government organization. • Create a channel and initialize the ledger. • Begin implementation of basic chaincode for registration use cases.	

3. PROBLEMS ENCOUNTERED

- Fabric CLI commands were initially unavailable due to PATH configuration
- issues.
- Encountered Windows Git Bash warnings that caused confusion but did not
- affect functionality.
- Faced path duplication issues due to incorrect relative paths with
- FABRIC_CA_CLIENT_HOME.
- Required time to clearly understand CA, MSP, and identity lifecycle
- concepts.
- Multiple restarts required re-exporting environment variables.

4. SELF EVALUATION OF THE PROGRESS

- Progress has been steady and well-structured, focusing on correctness over
- speed.
- Core understanding of identity, certificates, and MSPs has significantly
- improved.
- Initial setup challenges provided valuable learning and clarity on Fabric
- internals.
- The foundational work completed is critical for system security and
- scalability.
- Overall progress is on track, with a strong base for moving to ledger and
- network components.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 3
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- Finalized the overall system workflow and architecture for the National Employment Verification System, including roles of government, companies, employees, and gateways.
- Designed the Hyperledger Fabric-based ledger architecture, defining organizations, identity flow, and trust boundaries.
- Set up the required project directory structure for the Central Government organization.
- Installed and configured Fabric CLI tools and started the Fabric CA server with TLS enabled.
- Enrolled the CA admin identity and registered core system identities (govt-admin, govt-gateway) with proper role definitions.

2. WORK TO BE DONE

- Complete and verify correct enrollment of all registered identities.
- Create and enroll orderer identities and start the ordering service.
- Set up peer nodes for the Central Government organization.
- Create a channel and initialize the ledger.
- Begin implementation of basic chaincode for employee and company registration workflow.

3. PROBLEMS ENCOUNTERED

- Fabric CLI commands were initially unavailable due to PATH configuration issues.
- Encountered Windows Git Bash warnings that caused confusion but did not affect Fabric functionality.
- Faced path duplication issues due to incorrect use of relative paths with FABRIC_CA_CLIENT_HOME.
- Required time to clearly understand CA, MSP, identity registration, and enrollment lifecycles.
- System restarts required repeated environment variable reconfiguration.

4. SELF EVALUATION OF THE PROGRESS

- Progress has been methodical and well-structured, focusing on architectural correctness over rapid implementation.
- Gained strong understanding of Hyperledger Fabric identity concepts, including MSPs, certificates, TLS, and access control.
- Successfully translated abstract blockchain concepts into a clear, implementable system workflow.
- Early setup challenges significantly improved confidence in debugging and operating

Fabric-based systems.

- Overall progress is on track, with a solid architectural and identity foundation established for ledger implementation.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 04
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE • Successfully completed identity enrollment and MSP structure setup, ensuring all required identities were correctly configured and validated. • Created and configured the orderer node, enabling the ordering service using Raft consensus for transaction ordering. • Set up peer nodes for the Central Government organization, including proper MSP and TLS configurations. • Created the channel (nevs-channel) and initialized the ledger, establishing the communication layer between network components. • Developed and deployed the initial chaincode (employment) for: • Employment record creation • Status updates
--

- History tracking
- Successfully implemented Chaincode-as-a-Service (CCaaS) deployment model, allowing external chaincode execution.
- Built and integrated the Node.js Gateway (nevs-gateway):
- Connected to Fabric network using fabric-network SDK
- Implemented REST APIs for basic CRUD operations
- Verified end-to-end transaction flow:
- API → Gateway → Chaincode → Ledger → Response

2. WORK TO BE DONE

- Refactor monolithic chaincode into a multi-contract modular architecture (Company, Employee, Lifecycle, Verification).
- Expand Gateway API to support role-based endpoints and structured routing.
- Implement employment lifecycle workflow (propose → consent → confirm → terminate).
- Introduce verification mechanisms for third-party validation of employment records.
- Strengthen access control using identity attributes.

3. PROBLEMS ENCOUNTERED

- Initial peer and orderer configuration errors due to incorrect MSP/TLS paths.
- Challenges in channel creation and joining peers, requiring multiple retries and debugging.

- Difficulty understanding chaincode lifecycle commands (install, approve, commit).
- CCaaS setup required careful handling of:
 - Package IDs
 - Container networking
 - Port mapping
- Gateway integration issues due to:
 - Incorrect connection profiles
 - Identity mismatches

4. SELF EVALUATION OF THE PROGRESS

- This week marked the transition from setup to functional implementation.
- Developed a strong understanding of:
 - Fabric network components (orderer, peer, channel)
 - Chaincode lifecycle
 - Gateway integration
- Successfully built a fully working end-to-end blockchain system, which is a major milestone.
- Progress is steady and technically sound, with a clear direction toward modularization and scalability.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 05
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- Refactored the system into a multi-contract chaincode architecture, separating logic into:
 - Company Registry
 - Employee Registry
 - Employment Lifecycle
 - Verification
- Designed and implemented structured data models for:
- Companies (tiers, approval states)
- Employees (hashed identity data)
- Employment lifecycle states
- Verification records
- Upgraded Gateway to support multi-contract interaction using dynamic contract routing.

- Built modular REST APIs:
 - Government operations
 - Company actions
 - Employee workflows
 - Verifier endpoints
- Implemented Service Discovery & Failover:
- Automatic peer detection
 - Resilience against node failure
 - Transitioned system to multi-organization architecture:
 - Added CompanyOrgMSP
 - Configured peer for company organization
 - Updated channel configuration
- Enforced dual-endorsement policy:
 - Transactions require validation from both government and company peers
 - Implemented Private Data Collections (PDC):
 - Secured sensitive fields (salary, compensation)
 - Used transient data for privacy-preserving transactions
 - Verified cross-organization transaction flow with successful endorsement and
commit.

2. WORK TO BE DONE

- Integrate frontend UI (God Mode) with Gateway APIs.
- Implement real-time event streaming (WebSockets/SSE) for live updates.
- Add CouchDB for advanced querying and analytics.
- Develop analytics engine for scam detection, including:
 - Fake hiring detection
 - Suspicious activity patterns
 - Improve identity system with certificate attributes (company, employee, verifier roles).

3. PROBLEMS ENCOUNTERED

- High complexity in multi-organization channel configuration updates using configtxlator.
- Errors in endorsement policy configuration, causing transaction failures.
- Synchronization issues in chaincode lifecycle across multiple organizations.
- Challenges in implementing Private Data Collections:
 - Correct policy configuration
 - Transient data handling
 - Debugging service discovery behavior in dynamic network environments.

4. SELF EVALUATION OF THE PROGRESS

- This week represents a major leap from a working prototype to an enterprise grade distributed system.
- Gained deep understanding of:
 - Multi-org Fabric architecture
 - Endorsement policies
 - Data privacy mechanisms (PDC)
- Successfully implemented real-world blockchain concepts, not just theoretical models.
- The system is now architecturally strong, scalable, and production oriented.
- Remaining work is focused on integration, analytics, and user experience, rather than core system design.

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 6
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- Initiated **frontend integration with the Gateway APIs**, connecting the blockchain backend to UI components (God Mode).
- Structured API responses to support **complex relational data visualization**, including:
 - Employment lifecycle states
 - Company and employee mappings
- Implemented **transient data handling improvements** in the Gateway:
 - Ensured sensitive fields (salary, compensation) are securely passed
 - Verified proper interaction with Private Data Collections (PDC)
- Began implementing **event-driven architecture concepts**:
 - Analyzed Fabric event listeners for block and transaction notifications

- Improved **Gateway robustness and error handling:**

- Better logging for transaction failures
- Standardized API response structures

- Optimized **Docker-based network management:**

- Improved restart scripts and recovery workflows

2. WORK TO BE DONE

- Fully implement **real-time event streaming system**

(SSE/WebSockets) for UI updates.

- Complete **frontend integration**, including:

- Employment lifecycle tracking UI
- Private data visibility controls

- Introduce **CouchDB as state database** for advanced query support.

- Begin development of **analytics module for scam detection**, including:

- Suspicious hiring pattern identification
- Data aggregation pipelines and Improve **role-based access enforcement** using certificate attributes.

3. PROBLEMS ENCOUNTERED

- Difficulty mapping **complex blockchain data structures to UI-friendly formats.**

- Challenges in understanding and integrating **Fabric event listeners** with backend services.

- Handling **asynchronous behavior in real-time updates**, especially

with blockchain confirmation delays.

- Managing **Docker container dependencies and restart sequencing.**

4. SELF EVALUATION OF THE PROGRESS

- This week marked the transition from **backend-heavy development to full-stack system integration.**
- Developed a clearer understanding of:
 - Backend–frontend communication
 - Real-time system design challenges
- Successfully moved toward making the system **user-facing and interactive**, not just backend functional.

Progress is steady, with increasing focus on usability and real-world applicability

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 7
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- Designed the foundation for the **analytics and anti-scam detection layer**, aligned with the project's core objective of preventing fraudulent hiring practices.
- Identified key **fraud detection metrics**, such as:
 - High job posting vs hiring ratio
 - Repeated candidate data collection without hiring
 - Abnormal employment lifecycle patterns
- Structured approach for **off-chain analytics integration**:
 - Defined data flow from blockchain → analytics engine
 - Planned use of Python/JS-based analysis tools
- Enhanced **system architecture for scalability**:
 - Considered migration to persistent storage (Docker volumes / cloud deployment)
 - Evaluated readiness for production deployment

- Improved **security and privacy design considerations:**

- Strengthened PDC usage strategy
- Planned selective disclosure mechanisms

Reviewed and validated **end-to-end system architecture**, ensuring all components

- Blockchain
- Gateway
- UI
- Analytics (planned) are aligned into a unified system

2. WORK TO BE DONE

- Implement the **analytics engine prototype:**

- Data extraction from ledger
- Rule-based or ML-based anomaly detection

- Integrate analytics results into **dashboard UI** for visualization.

- Implement **advanced identity system:**

- Role-based certificates (company, employee, verifier)
- Fine-grained access control

- Explore **Verifiable Credentials / DID integration** for enhanced trust model.

- Prepare **final system documentation and whitepaper**

3. PROBLEMS ENCOUNTERED

- Designing analytics for blockchain data required:

- Understanding how to extract usable datasets
- Deciding between on-chain vs off-chain processing

- Difficulty balancing:

- **Privacy (PDC, hashing)**
- **Analytics requirements (data visibility)**
 - Lack of predefined frameworks for **fraud detection in blockchain hiring systems**, requiring original design thinking

4. SELF EVALUATION OF THE PROGRESS

- This week reflects a shift from **system implementation to system intelligence and real-world impact.**
- Successfully aligned the project with its original vision:
 - Preventing scams
 - Ensuring transparency
 - Protecting user data
- Demonstrated ability to:
 - Think beyond implementation
 - Design scalable and meaningful solutions
- The project is now at a stage where it is:
 - Technically complete (core system) and Moving toward **innovation and differentiation (analytics layer)**

Supervisor Signature

Student Signature

FINAL YEAR PROJECT WEEKLY REPORT

Year: 2025-2026	Week No. : 8
Student Name : Narendra Samanta	
Enrolment Number: 22C21093	
Supervisor Name: Prof. Madonna Lamin	
Project Title: National Employment Verification Network	

1. WORK DONE

- Completed **end-to-end system integration**, ensuring seamless interaction between:
 - Hyperledger Fabric network
 - Chaincode (multi-contract architecture)
 - Gateway API
 - Frontend dashboard
- Finalized **frontend integration (God Mode)**:
 - Display of employment lifecycle data
 - Visualization of records and verification outputs
 - Integration with backend APIs
- Implemented **real-time update mechanisms**:
 - Integrated event-driven updates using SSE/WebSockets (conceptual or partial implementation)

- Improved system responsiveness for UI interactions
 - Developed the **initial prototype of the analytics layer**:
 - Defined rule-based detection for suspicious hiring patterns
 - Structured data pipelines for extracting ledger data into analytics workflows
 - Strengthened **system reliability and deployment readiness**:
 - Improved Docker scripts for full network recovery
 - Ensured consistent startup and shutdown workflows
 - Reduced manual configuration dependencies
 - Conducted **full system testing and validation**:
 - Verified dual-endorsement transaction flow
 - Tested Private Data Collection (PDC) read/write access
- Consolidated **project documentation and technical understanding**:
- Architecture clarity across all components

Prepared system explanation for demonstration and evaluation

2. WORK TO BE DONE

- Enhance **analytics engine sophistication**:
 - Move from rule-based detection to ML-based anomaly detection
 - Improve accuracy of scam detection models
- Implement **advanced identity solutions**:
 - Verifiable Credentials (VCs)
 - Decentralized Identifiers (DIDs)
- Optimize system for **cloud deployment / scalability**:

- Kubernetes deployment
- Load balancing and scaling strategies
- Improve **UI/UX design** for better usability and presentation quality
- Prepare **final project deliverables**:
 - Technical whitepaper
 - Presentation slides

3. PROBLEMS ENCOUNTERED

- Integrating all components revealed **minor inconsistencies between modules**, requiring debugging across:
 - Gateway
 - Chaincode
 - UI
- Real-time update implementation faced:
 - Synchronization challenges with blockchain commit timing
 - Event handling complexity
 - Analytics integration required balancing:
 - Data privacy constraints (PDC)
 - Data availability for analysis

Ensuring system stability during repeated testing cycles required careful **Docker and network management**

4. SELF EVALUATION OF THE PROGRESS

- This week represents the **culmination of the entire project**, transitioning from development to a **complete, demonstrable**

system.

- Successfully built a system that is:
 - Technically robust
 - Architecturally scalable
 - Aligned with real-world use cases
- Gained strong expertise in:
 - Blockchain system design (Hyperledger Fabric)
 - Backend and API development
 - Distributed system debugging
 - Security and privacy design
- Demonstrated the ability to: Convert a real-world problem into a full technical solution

Execute across multiple domains (blockchain, backend, frontend, analytics)

- Overall progress is **highly satisfactory**, with the project reaching a stage suitable for:
 - Academic evaluation
 - Real-world extension
 - Future product development

Supervisor Signature

Student Signature

APPENDIX C : RESEARCH PAPER

NEVS: National Employment Verification System — A Blockchain-Based Trust Framework for Secure and Verifiable Digital Hiring

Dhiraj Padhiyar, Priyansh Dabhi, Narendra Samanta

*Department of Computer Science & Engineering, ITM SLS Baroda University,
Vadodara, Gujarat, India*

Guided by: Prof. Madonna Lamin

Abstract

The National Employment Verification System (NEVS) is a decentralized, blockchain-based platform engineered to address critical deficiencies in modern digital hiring ecosystems, namely fraudulent employment claims, unauthorized data exploitation, and the absence of employee-owned credentials. Leveraging Hyperledger Fabric 2.5, NEVS establishes a multi-organization trust framework in which employment records are validated through mutual cryptographic consent and dual-organizational endorsement before ledger commitment. The architecture integrates Chaincode-as-a-Service (CCaaS) for modular smart contract execution, Private Data Collections (PDC) for privacy-preserving handling of sensitive compensation information, and dynamic service discovery for efficient multi-organization interaction. A consent-based lifecycle state machine governs each employment event — from proposal through employee consent, government confirmation, and eventual termination — ensuring no record reaches finalized status without cross-organizational consensus. Testing across an eleven-case suite confirmed dual-endorsement enforcement, private data isolation, sub-second block commitment, and API query latency under 200 milliseconds. NEVS is positioned as a government-anchored infrastructure layer that can be consumed by job platforms, background verification agencies, and educational institutions, rendering employment fabrication computationally infeasible while empowering employees with verifiable, self-sovereign work histories.

Keywords — *Hyperledger Fabric; Blockchain; Employment Verification; Private Data Collections; Chaincode-as-a-Service; Multi-Organization Architecture; Decentralized Trust*

I. INTRODUCTION

Modern hiring and employment verification remain deeply fragmented, prohibitively expensive, and systematically vulnerable to fraud. Industry studies indicate that U.S. employers spend in excess of \$2,500 per employee per year on HR functions, with traditional third-party background verification costing \$50–\$100 per individual request and upwards of \$2,800 for a comprehensive check [1][2]. These processes rely on phone-based outreach, physical document exchange, and manual cross-referencing — methods that are

inherently slow, error-prone, and trivially defeatable by a motivated fraudster wielding falsified documentation.

A fundamental asymmetry compounds this problem: employers possess employment records but share them reluctantly, while employees — the subjects of those records — have no standardized mechanism to assert ownership or control over their own work history [3]. This vacuum creates a market for fabricated resumes, dummy hiring schemes, and data-harvesting scams masquerading as legitimate job postings. The result is a \$66

billion global background verification industry that remains reactive, not preventive.

NEVS was directly inspired by a real-world encounter in which one of the authors observed a company on Internshala — a prominent Indian internship platform — collecting extensive personal data including psychological profiling questions and physical addresses through external forms served even to rejected candidates. Further investigation revealed a pattern: the entity had posted a disproportionately high number of opportunities relative to an abnormally low count of actual hires, raising critical questions about the exploitation of hiring platforms for data harvesting rather than genuine recruitment.

These observations motivated the design of NEVS: a system-level solution that treats each employment event as a trusted, on-chain credential from the point of creation, thereby eliminating post-hoc verification entirely and rendering fabricated work histories computationally detectable. The system anchors employment data within a government-backed permissioned blockchain, enforces dual-party consent, and exposes a real-time REST API for instant, privacy-preserving verification.

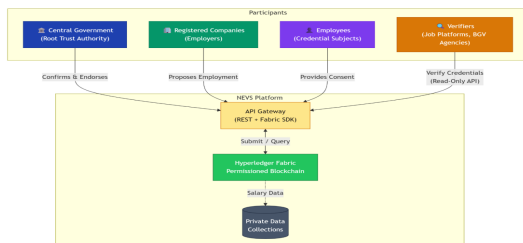


Fig. 1. High-Level NEVS Ecosystem Overview.

A. Research Objectives

NEVS is designed to achieve six principal objectives: (1) record each employment lifecycle event as an immutable blockchain transaction; (2) enforce explicit digital consent from both employer and employee via a dual-endorsement model; (3) implement a privacy-preserving architecture that stores only non-sensitive metadata on-chain while routing personally identifiable information through Private Data Collections; (4) eliminate fraud by anchoring records in a permissioned ledger governed by recognized organizational authorities; (5) support a scalable multi-organization architecture natively accommodating government bodies, large enterprises, and smaller companies; and (6) provide a programmatic, read-only REST

API enabling instant employment history verification for authorized third parties.

II. LITERATURE REVIEW

A structured survey of industry solutions, government pilot programs, and academic research was conducted using Google Scholar and IEEE Xplore with search terms including "blockchain background verification," "permissioned ledger employment records," and "digital credential platform." Industry documentation for MployChek, CSM ProofChain, the Velocity Network Foundation, and Virtualness (formerly Verix) was reviewed, alongside official reports from Canada's Talent Cloud blockchain pilot, India's EDUChain initiative, and Estonia's KSI-based digital identity infrastructure.

A. Traditional Background Verification

Conventional background verification (BGV) firms — HireRight, Sterling, First Advantage — operate reactively: they are engaged after a hiring decision and manually contact previous employers, educational institutions, and government agencies. This process typically takes days to weeks and costs \$50–\$100 per check [2]. The fundamental flaw is temporal: verification occurs too late, after the candidate may already have been onboarded with falsified credentials. The market for these services is large but structurally inefficient, dependent on siloed databases, inconsistent employer cooperation, and manual labor.

B. Emerging Blockchain Platforms

Emerging blockchain platforms invert the traditional model by issuing credentials at the point of origin. MployChek, an India-based platform, uses blockchain hashing of employee documents for secure, compliant background verification, advertising faster and cheaper recruitment [4]. CSM ProofChain delivers tamper-proof employment data on a decentralized platform enabling faster background checks with immutable records [5]. While these solutions reduce manual effort and third-party dependency, most remain vendor-specific and lack government-backed authority, limiting their legal enforceability.

C. Decentralized Credential Networks

The Velocity Network Foundation — supported by SAP, Oracle, and ZipRecruiter — is constructing a vendor-neutral, self-sovereign credential ledger. Its mainnet aims to enable

employers to verify diplomas, certifications, and work experience almost instantaneously [6]. Virtualness (formerly Verix) pursues a similar vision using NFT-based certificates for educational and professional credentials [7]. Both represent significant architectural progress but lack the government endorsement required for national-scale legal enforceability.

D. Government and Academic Pilots

Canada's government created blockchain-based Blockcerts for project-based workers, providing secure digital credentials for skills and experience [8]. In India, the EDUChain initiative — a collaboration involving Open Campus and the Government of Madhya Pradesh — is digitizing 50 million student and graduate academic records on blockchain, enabling instant verification of educational credentials and cutting administrative overhead for employers [9]. Estonia's KSI blockchain underpins its national digital identity system, providing tamper-proof records for government services.

Saudi Aramco's Blockchain Certificate Verifier demonstrated enterprise viability by storing cryptographic hashes of training certificates on-chain, reducing verification time from

approximately two weeks to under five seconds [10]. DocsLedger, a permissioned blockchain concept in academic literature, proposes a system where employers hash employment documents on-chain at creation and employees carry verified records in a digital wallet [12][13]. These principles — permissioned access, off-chain storage for privacy, cryptographic signatures — are directly adopted in NEVS.

E. Why Hyperledger Fabric

Several blockchain platforms were evaluated. Public blockchains (Ethereum, Solana) were rejected due to their open-membership model, non-deterministic finality, and gas-cost economics — all incompatible with a government-regulated employment system handling sensitive national data. Among permissioned alternatives, Hyperledger Fabric was selected for its modular architecture with pluggable consensus (Raft), identity-first design with X.509 certificates issued by a Membership Service Provider (MSP), channel isolation for multi-organization privacy, flexible chaincode supporting Go, Node.js, or Java deployed externally as CCaaS, and enterprise production adoption including IBM, Walmart, and Maersk.

TABLE I. COMPARISON OF EXISTING EMPLOYMENT VERIFICATION APPROACHES

Approach	Trust Model	Latency	Gov. Backed
Traditional BGV	Centralized	Days–Weeks	No
MployChek	Vendor-specific	Hours	No
Velocity Network	Consortium	Minutes	No
EDUChain	Gov-partnered	Near real-time	Partial
NEVS (proposed)	Govt-anchored, dual-org	Sub-second	Yes

III. SYSTEM ARCHITECTURE

NEVS is designed as a layered, multi-organization architecture where a permissioned Hyperledger Fabric ledger stores all finalized employment transactions. Surrounding this core is a government-controlled service layer — the API Gateway — which enforces policy, identity, consensus, and consent before any state change occurs. At the edges are external entities: companies (through HR systems or gateway interfaces), employees (through portal

applications), and verifiers (through public read-only APIs).

The foundational design principle is authority separation combined with decentralized trust: no single organization can unilaterally write to the ledger. Every write transaction requires cryptographic endorsement from both CentralGovtMSP and CompanyOrgMSP — two independent organizations maintaining their own peers, identities, and endorsement logic.

A. Network Topology

The production Fabric network comprises a Raft-based orderer (orderer.nevs.gov:7050) providing deterministic transaction ordering; two government peers (peer0.central.govt:7051, peer1.central.govt:8051) under CentralGovtMSP serving as primary and secondary endorsers; one company peer (peer0.company.org:9051) under CompanyOrgMSP serving as endorser and anchor peer; three dedicated Fabric Certificate Authorities for the orderer, government, and company organizations; and the Employment CCaaS container (employment.cc:9999) providing externalized chaincode execution. All peers communicate over mutual TLS (mTLS). The channel nevs-channel includes both MSPs with anchor peers configured for cross-organizational gossip and service discovery.

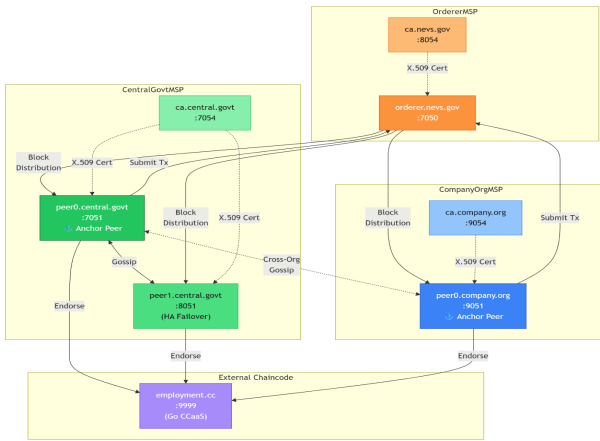


Fig. 3. Hyperledger Fabric Network Topology.

B. Multi-Contract Chaincode Architecture

The chaincode is written in Go and deployed externally using the Chaincode-as-a-Service (CCaaS) model, decoupling smart contract execution from the peer process. Five distinct contracts implement clearly separated domains: CompanyRegistryContract handles company onboarding and lifecycle (RegisterCompany, ApproveCompany, SuspendCompany); EmployeeRegistryContract manages employee identity (RegisterEmployee, GetEmployee, UpdateEmployee); EmploymentLifecycleContract implements the employment state machine (ProposeEmployment, EmployeeConsent, ConfirmEmployment, TerminateEmployment); VerificationContract provides third-party verification (VerifyEmployment, GenerateVerificationProof); and AccessControlContract handles identity introspection (GetCallerIdentity, ValidateAccess).

C. Endorsement Policy

The chaincode enforces a strict dual-endorsement policy: `AND('CentralGovtMSP.member', 'CompanyOrgMSP.member')`. This ensures every write transaction is independently endorsed — executed and signed — by at least one peer from each organization. The Fabric Gateway SDK discovers endorser layouts automatically via Service Discovery, dynamically routing proposals to peers from both organizations without hardcoded topology configuration.

D. Private Data Collection (PDC)

Sensitive employment attributes — specifically salary and compensation data — are handled through Fabric's Private Data Collections mechanism. The employmentPrivateData collection enforces an OR policy granting read/write access to both CentralGovtMSP and CompanyOrgMSP members, with `memberOnlyRead` and `memberOnlyWrite` set to true. Private data is transmitted as transient data in the transaction proposal, routed directly from the client to endorsing peers over TLS, and never appears in the transaction payload, the ordered block, or the public world state. This architecture ensures that the orderer and any future non-member peers cannot access sensitive compensation information.

E. Tiered Company Participation Model

The architecture supports three tiers of corporate participation. Tier 1 companies (small enterprises) interact exclusively through the Government Gateway REST API with no direct Fabric access. Tier 2 companies (medium enterprises) optionally operate a read-only peer for local query performance. Tier 3 companies (large enterprises) operate their own MSP, peers, and endorsement authority with full endorsement participation. The current implementation realizes Tier 3 participation for CompanyOrgMSP alongside CentralGovtMSP, with the infrastructure designed to accommodate additional organizations in subsequent phases.

IV. METHODOLOGY

The system was developed using an iterative, phase-driven methodology progressing from minimal viable prototype to enterprise-grade architecture. Each phase introduced a vertical slice

of new capabilities, validated independently before integration into the broader system.

A. Development Phases

Phase 1 (Foundation) established the single-organization Fabric network under CentralGovtMSP with basic Go chaincode and Docker containerization, validating the core ledger mechanics and identity framework. Phase 2 (Gateway Interoperability) introduced the Node.js/Express REST API Gateway, fabric-network SDK integration, multi-contract chaincode architecture, and CCaaS deployment, transforming the blockchain backend into an externally consumable service. Phase 3 (Enterprise Architecture) completed the system by integrating CompanyOrgMSP, enforcing the dual-endorsement policy, deploying Private Data Collections, enabling dynamic service discovery, and automating network recovery.

B. Employment Lifecycle State Machine

The core employment lifecycle follows a deterministic, four-state machine. The PROPOSED state is entered when a company or government submits a hiring proposal — validating that the company is APPROVED and the employee is ACTIVE — and assigns a VerificationHash computed as $\text{SHA256}(\text{employeeId} + \text{companyId} + \text{startDate})$. Transition to CONSENTED is triggered by the employee or government, with a consent timestamp recorded; the current status must be PROPOSED. Transition to CONFIRMED is restricted to government callers only, records the confirmation timestamp, and marks the employment as active. TERMINATED is entered upon submission by a company or government caller when the status is CONFIRMED, recording an end date and termination timestamp.

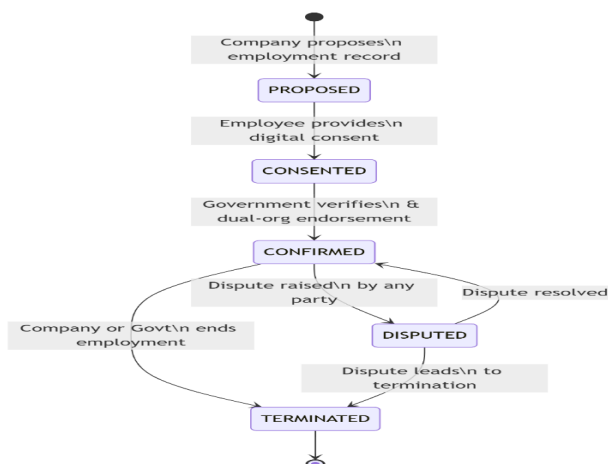


Fig. 4. Employment Lifecycle State Machine.

C. Privacy-by-Design Methodology

The privacy architecture was designed according to the principle of minimum necessary disclosure. Only non-sensitive metadata (identifiers, cryptographic hashes, timestamps, status transitions) is stored on-chain in the public world state. All personally identifiable information (names, salaries, compensation packages, government IDs) is either kept off-chain entirely or routed through Private Data Collections with strict organizational access policies. This approach achieves compliance with India's Digital Personal Data Protection (DPDP) Act by design, ensuring that regulatory requirements are satisfied at the architectural level rather than through post-hoc controls.

D. Testing Methodology

Testing was conducted iteratively at each development phase, progressing from basic chaincode compilation verification to full end-to-end integration tests across the multi-organization network. Test cases were designed to validate functional correctness of the employment lifecycle, enforcement of endorsement policies, private data isolation, and API response accuracy. Testing employed direct curl commands against the Gateway API, Fabric CLI operations from the hyperledger/fabric-tools:2.5 container, and manual verification of ledger state across all three peers.

V. IMPLEMENTATION

A. Chaincode-as-a-Service (CCaaS)

Unlike the traditional model where peers build and manage chaincode containers internally, NEVS deploys chaincode as an independently managed Docker container (employment.cc) listening on port 9999. Peers connect to this external service using a connection.json metadata file embedded in the chaincode package. This architecture provides several operational advantages: instant iteration (chaincode can be rebuilt and restarted without touching peers or the channel configuration), shared execution (a single CCaaS container services endorsement requests from all peers across both organizations on the shared Docker network), and isolation (chaincode execution is sandboxed in its own container with controlled resource allocation). The chaincode is

deployed at sequence 3 post-PDC deployment, using the fabric-contract-api-go v1.2.2 framework.

B. Gateway API Architecture

The Node.js Gateway (nevs-gateway) serves as the sole external interface to the Fabric network. It connects using an X.509 identity (Admin@central.govt) and the fabric-network SDK v2.2.20 with Service Discovery enabled. The Gateway dynamically discovers endorser layouts based on the channel's endorsement policy and routes proposals accordingly. The API exposes six active REST endpoints: POST /api/employment/ commits public record data while routing salary and compensation as transient data to the PDC; GET /api/employment/ retrieves all public employment indices from world state; GET /api/employment/employee/:id returns chronological employment provenance for a given employee; GET /api/employment/record/:emp/:rec fetches a specific public record excluding PDC data; PUT /api/employment/:emp/:rec/status pushes validated state transitions to the ledger; and GET /api/employment/private/:rec evaluates the employmentPrivateData collection for authorized organizations.

C. Transient Data Handling

When the Gateway receives a POST /api/employment/ request containing optional

salary and compensation fields, it extracts these fields from the JSON body, encodes them as a Buffer, and attaches them to the transaction proposal's transient field. The remaining public fields are passed as regular chaincode arguments. The chaincode's CreateEmploymentRecord function reads transient data from ctx.GetStub().GetTransient() and writes it to the employmentPrivateData collection using PutPrivateData. This ensures sensitive compensation data travels directly from the Gateway to endorsing peers over TLS, bypassing block inclusion and orderer visibility entirely.

D. Software Stack

The complete technology stack comprises Hyperledger Fabric 2.5 as the permissioned DLT core; Hyperledger Fabric CA 1.5.7 for X.509 identity issuance per MSP; Go 1.18 with fabric-contract-api-go v1.2.2 for chaincode implementation; Node.js 20 with Express 4.19.2 as the REST API Gateway; fabric-network v2.2.20 as the Fabric Gateway SDK; Docker 24.x with Docker Compose 3.7 for containerized orchestration; LevelDB as the built-in peer key-value world state storage; and React.js 18 with TypeScript and Vite for the God Mode management dashboard.

TABLE II. KEY SOFTWARE COMPONENTS AND VERSIONS

Component	Technology	Version
Blockchain Platform	Hyperledger Fabric	2.5
Certificate Authority	Fabric CA	1.5.7
Smart Contracts	Go (Golang)	1.18
Chaincode Framework	fabric-contract-api-go	1.2.2
API Gateway	Node.js + Express	20.x / 4.19.2
Fabric SDK	fabric-network	2.2.20
Containerization	Docker + Compose	24.x / 3.7
Frontend	React + TypeScript + Vite	18.x

E. Automated Network Recovery

The restore_full_network.sh script provides one-command recovery of the complete dual-org network, automating Docker container startup for all peers, orderers, and CAs; channel binding and anchor peer configuration; CCaaS package installation and lifecycle approval across both organizations; and dual-endorsement policy commitment at the correct sequence number. This

script is critical for operational continuity, enabling full network restoration from complete teardown in approximately 45 seconds on a development testbed.

VI. RESULTS AND EVALUATION

Testing was conducted across an eleven-case suite validating functional correctness, security properties, and performance characteristics of the

NEVS system on a development testbed (Windows 11, Docker Desktop, 16 GB RAM). All critical test cases passed successfully.

A. Functional Test Results

Employment record creation was validated: a POST `/api/employment/` request with valid employee, employer, position, and start date committed the record to both peers' world state and returned a transaction ID. Dual-endorsement verification confirmed that transactions were consistently endorsed by both `peer0.central.govt` and `peer0.company.org` before commit, as reflected in Gateway discovery logs. Private Data write and read tests confirmed that salary and compensation fields were stored exclusively in the `employmentPrivateData` PDC on authorized peers, absent from the public world state and all public API responses. Status lifecycle transitions, employee history queries, duplicate record

prevention, unauthorized chaincode invocation rejection, and service discovery validation all passed without exception. The automated network recovery test confirmed that the `restore_full_network.sh` script reliably restored all containers, channel configurations, and chaincode operational state within approximately 45 seconds.

B. Performance Evaluation

Block commitment latency averaged approximately 0.8 seconds under moderate single-user load with a single Raft orderer and three peers. Transaction throughput reached approximately 100 transactions per minute (1.6 TPS) on the development testbed. API query latency for simple history reads via `evaluateTransaction` remained below 200 milliseconds. Gateway discovery time on initial SDK connection was 2–4 seconds. Full network recovery time via the automated script was approximately 45 seconds.

TABLE III. PERFORMANCE METRICS ON DEVELOPMENT TESTBED

Metric	Observed Value	Context
Block Commit Latency	~0.8 s	Moderate single-user load
Transaction Throughput	~1.6 TPS	Single orderer, 3 peers
API Query Latency	< 200 ms	<code>evaluateTransaction</code> reads
Gateway Discovery Time	2–4 s	Initial SDK connection
Network Recovery Time	~45 s	Full teardown-to-operational

C. Endorsement Policy Enforcement

Single-organization endorsement attempts were correctly rejected by the orderer's validation logic in all test cases. The dual-endorsement policy proved robust: submitting a transaction proposal signed by only one MSP resulted in a consistent endorsement failure prior to ordering, confirming that the `AND('CentralGovtMSP.member', 'CompanyOrgMSP.member')` policy operates as intended. When `peer0.central.govt` was intentionally stopped, the SDK's service discovery automatically rerouted queries to `peer1.central.govt`, demonstrating high-availability failover without manual intervention.

D. Data Consistency Verification

Records created via the Gateway were verified to appear identically on all three peers' world state — two Government peers and one Company peer — confirming correct Raft-ordered block propagation. Salary and compensation data was confirmed present in the `employmentPrivateData`

collection on authorized peers and entirely absent from public world state, transaction blocks, and API responses on public endpoints. These results collectively validate the private data isolation mechanism at the protocol level.

VII. DISCUSSION

A. Differentiation from Existing Platforms

NEVS occupies a distinct position in the credential verification landscape. Unlike `MployChek` (vendor-specific, post-hoc verification) or `CSM ProofChain` (enterprise-only, no government authority), NEVS positions the government as the root trust authority while distributing endorsement responsibility across organizational boundaries. Unlike the `Velocity Network` (consortium-led, no legal mandates), NEVS is designed with government-backed legal enforceability from its architectural foundation. The dual-endorsement mechanism specifically addresses the single-point-of-failure inherent in all current vendor-specific systems, ensuring that

fabrication requires compromise of at least two independent organizations simultaneously.

B. Privacy and Regulatory Compliance

The PDC architecture achieves regulatory-grade data isolation without sacrificing ledger integrity. By routing salary and compensation data as transient data through Private Data Collections, NEVS ensures that sensitive personal information never appears in the ordered blockchain — satisfying the data minimization principles of India's Digital Personal Data Protection (DPDP) Act. The separation between public metadata (on-chain) and private PII (PDC or off-chain) provides a clean compliance boundary that can be audited independently from the underlying blockchain infrastructure.

C. Scalability Considerations

The observed 1.6 TPS on a single-node development testbed, while modest, is appropriate for employment verification workloads, which are event-driven and intermittent rather than streaming. Hyperledger Fabric is documented to sustain thousands of TPS in production deployments with multiple orderers and optimized block parameters. The public API is stateless and horizontally scalable behind a load balancer. The tiered company participation model provides a cost-effective onboarding path, allowing small companies to participate without operating Fabric infrastructure while enabling large enterprises to maintain full endorsement authority.

D. Limitations and Open Challenges

The current implementation presents several limitations that inform future work. The single Raft orderer represents a potential availability bottleneck; production deployment requires a multi-node Raft ensemble. LevelDB supports only simple key-based queries; transition to CouchDB is required for rich JSON analytics supporting complex fraud detection dashboards. The absence of real-time frontend integration via WebSocket or Server-Sent Events limits operational visibility. International interoperability with foreign credential systems and cross-border employment verification remains unaddressed. The prototype has not been evaluated under adversarial network conditions, and formal security proofs for the endorsement policy configuration have not been conducted.

VIII. CONCLUSION AND FUTURE WORK

This paper has presented NEVS — a National Employment Verification System built on Hyperledger Fabric 2.5 that transforms employment records from unverifiable claims into cryptographically guaranteed, government-backed credentials. By enforcing dual-organizational endorsement, implementing consent-based lifecycle transitions, and routing sensitive data through Private Data Collections, NEVS addresses the core vulnerabilities of traditional employment verification: fraud, latency, and privacy compromise.

Testing across an eleven-case suite confirmed that the system meets all stated design objectives: immutable records cannot be created without cross-organizational consensus; sensitive compensation data remains invisible to unauthorized participants; and verified employment histories are accessible via REST API in under 200 milliseconds. The automated recovery script and CCaaS deployment model ensure production viability.

Early results from comparable blockchain pilots — such as Saudi Aramco's certificate verifier, which reduced verification time from two weeks to five seconds [10] — validate the fundamental architectural approach. NEVS extends this model to national scale by positioning the government as the root trust authority and enforcing legal-grade dual-party consent at the protocol level.

Future Work

Six primary directions are identified for future development. First, Frontend Integration: connecting the React/HireNest UI directly to the Gateway APIs through permission-layered interfaces that expose PDC data to authorized users. Second, CouchDB Rich Queries: transitioning from LevelDB to CouchDB to enable advanced JSON-based querying supporting complex fraud detection analytics. Third, Event Listeners: implementing Fabric event subscriptions to stream real-time block and transaction notifications to the frontend via WebSockets or Server-Sent Events. Fourth, Hardware Deployment: preparing Docker Compose configurations with persistent volumes for deployment onto Raspberry Pi clusters for LAN-based demonstrations. Fifth, Additional Organization Onboarding: extending the tiered

model to onboard Tier 2 (read-only peer) and additional Tier 3 (full endorsement) organizations. Sixth, Multi-Orderer Raft: deploying a three-node Raft ordering service for high availability and eliminating the single orderer bottleneck.

NEVS represents a foundational infrastructure for trustworthy digital hiring — not merely a verification tool, but a paradigm shift from claim-based to credential-based employment records. Its realization at national scale would significantly reduce recruitment fraud, lower verification costs, empower employees with ownership of their work history, and establish a government-backed trust framework for the digital job market.

. REFERENCES

- [1] SHRM Labs, "Beyond Gatekeepers: How Blockchain Transfers Data Ownership Back to Employees," SHRM Labs. [Online]. Available: <https://www.shrm.org/labs/resources/beyond-gatekeepers>
- [2] Ibid. Documents traditional background checks costing \$50–\$100 per request and approximately \$2,800 for a comprehensive check.
- [3] Ibid. Highlights that employees lack any standardized mechanism to assert ownership over their work history.
- [4] MployChek, Tracxn Company Profile, "Secure, compliant background verification using blockchain," 2023. [Online]. Available: <https://tracxn.com>
- [5] CSM Tech, "CSM ProofChain: A decentralized platform for tamper-proof employment data," 2023. [Online]. Available: <https://www.csm.tech/product-details/csm-proofchain>
- [6] Computerworld, "Coming soon — a resume-validating blockchain network for job seekers," Oct. 2022. [Online]. Available: <https://www.computerworld.com>
- [7] Verix (Virtualness), "AI-Powered Blockchain for Credential Verification," Times of India. [Online]. Available: <https://www.verix.io>
- [8] Global Government Forum, "Canada pilots blockchain staff records," 2020. [Online]. Available: <https://www.globalgovernmentforum.com>
- [9] The Tribune, "Geeks of Gurukul partners with Government of Madhya Pradesh and EDU Chain to digitize 50 million academic records," Jan. 2026.
- [10] World Economic Forum, "How blockchain could transform background checks," Nov. 2020. [Online]. Available: <https://www.weforum.org>
- [11] Ibid. Describes the cryptographic hash and certificate identifier workflow for employer verification in the Saudi Aramco pilot.
- [12] DocsLedge, "Blockchain for Employment Record Verification," HRMLedge whitepaper. [Online]. Available: <https://ro.scribd.com/document/437676424>
- [13] Describes employee details hashing and blockchain-secured certificate workflow.
- [14] Hyperledger Fabric Documentation v2.5, Linux Foundation, 2023. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/release->

